

9. JVM

JVM

课程目标：

1. 了解 JVM 的发展史
2. 了解 JVM 运行原理
3. 掌握 JVM 基本组成
4. 掌握 JVM 垃圾回收算法
5. 掌握类加载机制
6. 掌握 JMM

版本更新内容：

1. JVM 运行时数据区所有部分的作用做了一个说明（解决了为什么需要这些区域的问题）补充了一些图片；
2. 方法区的实现：永久代/元空间举例说明（汽车的动能提供装置），添加了图片。
3. 添加图片：JVM 演示内存溢出时，Idea 设置的图片。
4. 新增垃圾收集器的作用，以及为什么有那么多的垃圾收集器的原因。
5. 一个对象的一生，JVM 总结和执行流程图。
6. 破坏双亲委派模型的案例（SPI->JDBC）

1.JVM 简介

JVM 是 Java Virtual Machine 的简称，意为 Java虚拟机。

虚拟机是指通过软件模拟的具有完整硬件功能的、运行在一个完全隔离的环境中的完整计算机系统。

常见的虚拟机：JVM、VMwave、Virtual Box。

JVM 和其他两个虚拟机的区别：

1. VMwave与VirtualBox是通过软件模拟物理CPU的指令集，物理系统中会有很多的寄存器；
2. JVM则是通过软件模拟Java字节码的指令集，JVM中只是主要保留了PC寄存器，其他的寄存器都进行了裁剪。

JVM 是一台被定制过的现实当中不存在的计算机。

1.1 JVM 发展史

1.Sun Classic VM

早在1996年Java1.0版本的时候，Sun公司发不了一款名为Sun Classic vm的java虚拟机，它同时也是世界上第一款商业java虚拟机，jdk1.4 时完全被淘汰。

这款虚拟机内部只提供解释器。

如果使用JIT编译器，就需要进行外挂。但是一旦使用了JIT编译器，JIT就会接管虚拟机的执行系统。解释器就不再工作。解释器和编译器不能配合工作。

现在Hotspot内置了此虚拟机；

2.Exact VM

为了解决上一个虚拟机问题，jdk1.2时，sun提供了此虚拟机。

Exact 具备现代高性能虚拟机的雏形，包含了一下功能：

1. 热点探测（将热点代码编译为字节码加速程序执行）；
2. 编译器与解析器混合工作模式。

只在Solaris平台短暂使用，其他平台上还是 classic vm

英雄气短，终被Hotspot虚拟机替换。

3.HotSpot VM

HotSpot 历史

1. 最初由一家名为“Longview Technologies”的小公司设计；
2. 1997年，此公司被Sun收购；2009年，Sun公司被甲骨文收购。
3. JDK1.3时，HotSpot VM成为默认虚拟机

目前 HotSpot 占用绝对的市场地位，称霸武林。

不管是现在仍在广泛使用JDK6，还是使用比较多的JDK8中，默认的虚拟机都是HotSpot；Sun/Oracle JDK和OpenJDK的默认虚拟机。从服务器、桌面到移动端、嵌入式都有应用。

名称中的HotSpot指的就是它的热点代码探测技术。它能够通过计数器找到最具编译价值的代码，触发即时编译(JIT)或栈上替换；通过编译器与解释器协同工作，在最优化的程序响应时间与最佳执行性能中取得平衡。

4.JRockit

JRockit 是专注于服务器端应用，目前在HotSpot的基础上，移植JRockit的优秀特性。

它可以不太关注程序的启动速度，因此JRockit内部不包含解析器实现，全部代码都靠即时编译器编译后执行；

大量的行业基准测试显示，JRockit JVM是世界上最快的JVM。

使用JRockit产品，客户已经体验到了显著的性能提高（一些超过了70%）和硬件成本的减少（达50%）；

优势：全面的Java运行时解决方案组合。

JRockit面向延迟敏感型应用的解决方案JRockit Real Time提供以毫秒或微秒级的JVM响应时间，适合财务、军事指挥、电信网络的需要；

MissionControl服务套件，它是一组以极低的开销来监控、管理和分析生产环境中的应用程序的工具；

2008，BEA被Oracle收购。

Oracle表达了整合两大优秀虚拟机的工作，大致在JDK8中完成。整合的方式是在HotSpot的基础上，移植JRockit的优秀特性。

5.J9 JVM

全称：IBM Technology for Java Virtual Machine，简称IT4J，内部代号：J9。

市场定位于HotSpot接近，服务器端、桌面应用、嵌入式等多用途JVM，广泛用于IBM的各种Java产品。

目前，有影响力的三大商用虚拟机之一，也号称是世界上最快的Java虚拟机(在IBM自己的产品上稳定)；

2017年左右，IBM发布了开源 J9 VM，命名 OpenJ9，交给Eclipse基金会管理，也称为Eclipse OpenJ9。

6.Taobao JVM（国产研发）

由AliJVM团队发布。阿里，国内使用Java最强大的公司，覆盖云计算、金融、物流、电商等众多领域，需要解决高并发、高可用、分布式的复合问题。有大量的开源产品。

基于OpenJDK 开发了自己的定制版本AlibabaJDK,简称AJDK。是整个阿里JAVA体系的基石；

基于OpenJDK HotSpot JVM发布的国内第一个优化、深度定制且开源的高性能服务器版Java虚拟机，它具有以下特点（了解即可）：

1. 创新的GCIH(GC invisible heap)技术实现了off-heap，即将生命周期较长的Java对象从heap中移到heap之外，并且GC不能管理GCIH内部的Java对象，以此达到降低GC的回收评率和提升GC的回收效率的目的。
2. GCIH中的对象还能够在多个Java虚拟机进程中实现共享。
3. 使用crc32指令实现JVM intrinsic降低JNI的调用开销；
4. PMU hardware的Java profiling tool和诊断协助功能；
5. 针对大数据场景的ZenGC。

taobao JVM应用在阿里产品上性能高，硬件严重依赖intel的cpu,损失了兼容性，但提高了性能，目前已经在淘宝、天猫上线，把Oracle官方JVM版本全部替换了。

1.2 JVM 和《Java虚拟机规范》

以上的各种 JVM 版本，比如 HotSpot 和 J9 JVM，都可以看做是不同厂商实现 JVM 产品的具体实现，而它们（JVM）产品的实现必须要符合《Java虚拟机规范》，《Java虚拟机规范》是 Oracle 发布 Java 领域最重要和最权威的著作，它完整且详细的描述了 JVM 的各个组成部分。

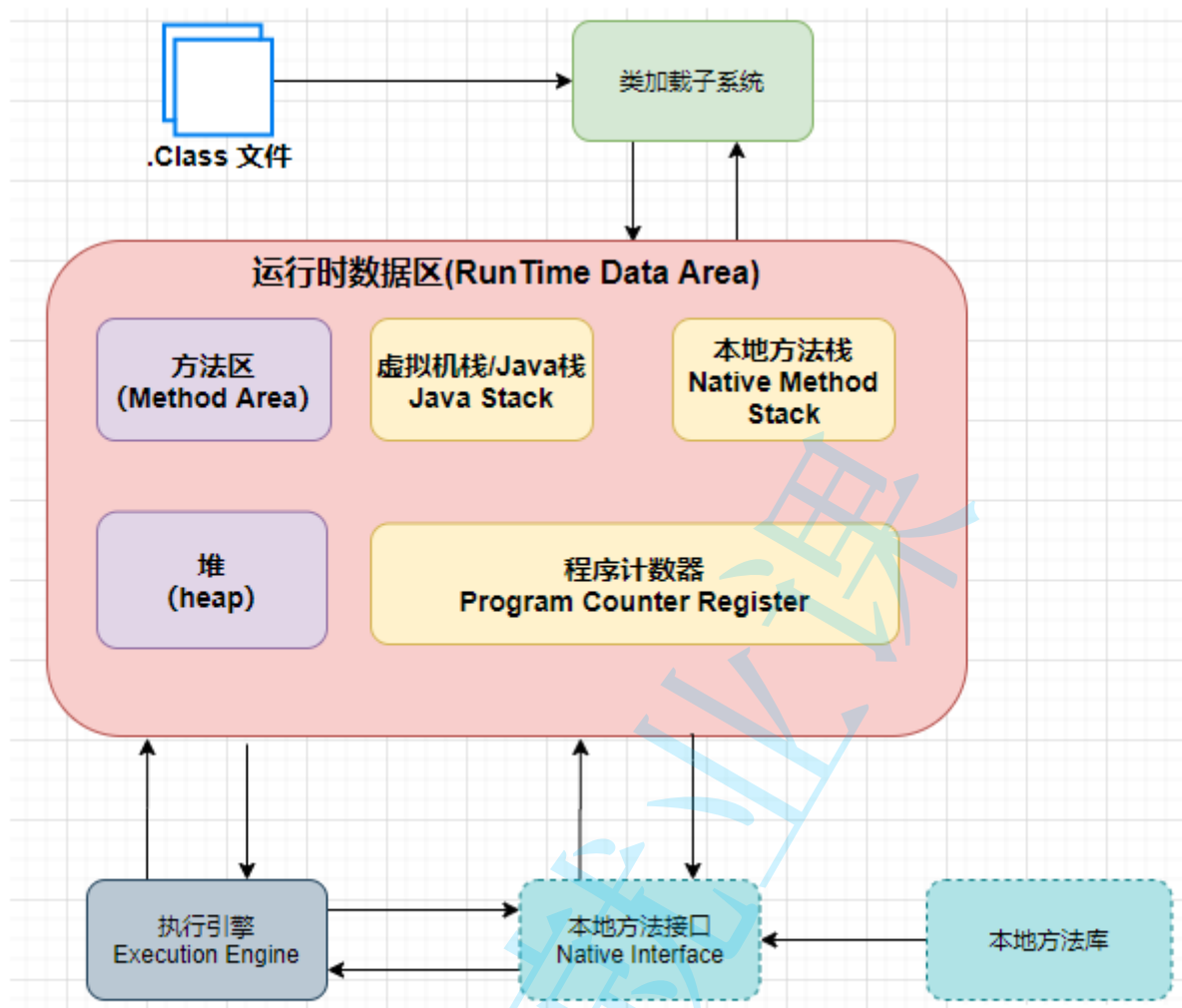
PS：本文以下部分，默认都是使用 HotSpot，也就是 Oracle Java 默认的虚拟机为前提来进行介绍的。

2. JVM 运行流程

JVM 是 Java 运行的基础，也是实现一次编译到处执行的关键，那么 JVM 是如何执行的呢？

JVM 执行流程

程序在执行之前先要把java代码转换成字节码（class文件），JVM 首先要把字节码通过一定的方式 **类加载器（ClassLoader）** 把文件加载到内存中 **运行时数据区（Runtime Data Area）**，而字节码文件是 JVM 的一套指令集规范，并不能直接交给底层操作系统去执行，因此需要特定的命令解析器 **执行引擎（Execution Engine）** 将字节码翻译成底层系统指令再交由CPU去执行，而这个过程中需要调用其他语言的接口 **本地库接口（Native Interface）** 来实现整个程序的功能，这就是这4个主要组成部分的职责与功能。

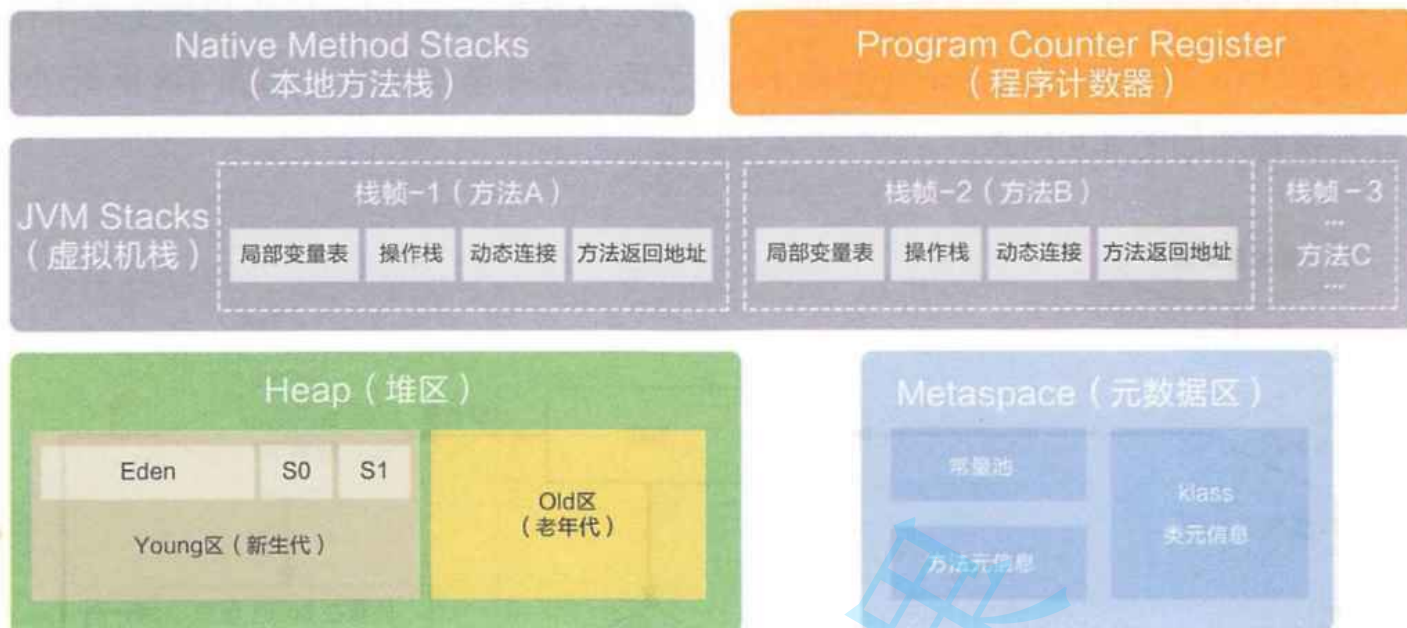


总结来看，JVM 主要通过分为以下 4 个部分，来执行 Java 程序的，它们分别是：

1. 类加载器 (ClassLoader)
2. 运行时数据区 (Runtime Data Area)
3. 执行引擎 (Execution Engine)
4. 本地库接口 (Native Interface)

3. JVM 运行时数据区

JVM 运行时数据区域也叫内存布局，但需要注意的是它和 Java 内存模型 ((Java Memory Model, 简称 JMM) 完全不同，属于完全不同的两个概念，它由以下 5 大部分组成：



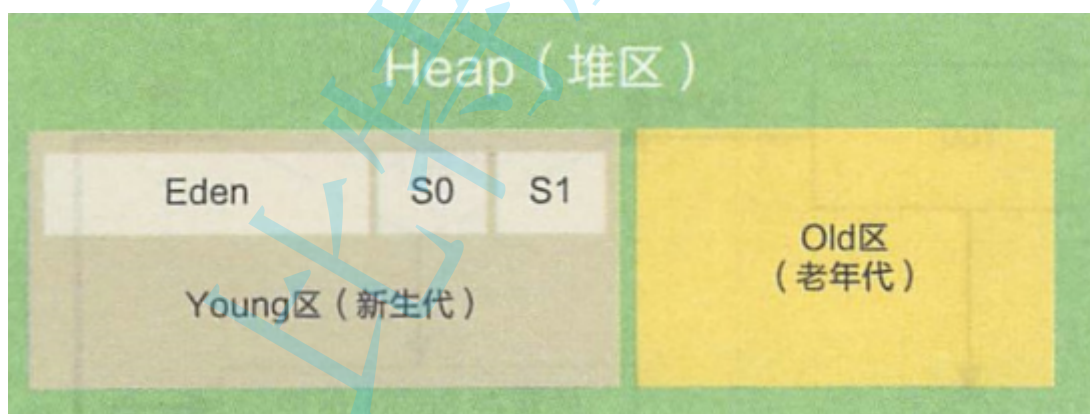
3.1 堆（线程共享）

堆的作用：程序中创建的所有对象都在保存在堆中。

我们常见的 JVM 参数设置 -Xms10m 最小启动内存是针对堆的，-Xmx10m 最大运行内存也是针对堆的。

ms 是 memory start 简称，mx 是 memory max 的简称。

堆里面分为两个区域：新生代和老年代，新生代放新建的对象，当经过一定 GC 次数之后还存活的对象会放入老年代。新生代还有 3 个区域：一个 Endn + 两个 Survivor (S0/S1)。



垃圾回收的时候会将 Endn 中存活的对象放到一个未使用的 Survivor 中，并把当前的 Endn 和正在使用的 Survivor 清楚掉。

3.2 Java虚拟机栈（线程私有）

Java 虚拟机栈的作用：Java 虚拟机栈的生命周期和线程相同，Java 虚拟机栈描述的是 Java 方法执行的内存模型：每个方法在运行的同时都会创建一个栈帧（Stack Frame）用于存储局部变量表、操作数栈、动态链接、方法出口等信息。咱们常说的堆内存、栈内存中，栈内存指的就是虚拟机栈。

Java 虚拟机栈中包含了以下 4 部分：



1. 局部变量表：存放了编译器可知的各种基本数据类型(8大基本数据类型)、对象引用。局部变量表所需的内存空间在编译期间完成分配，当进入一个方法时，这个方法需要在帧中分配多大的局部变量空间是完全确定的，在执行期间不会改变局部变量表大小。简单来说就是存放方法参数和局部变量。
2. 操作栈：每个方法会生成一个先进后出的操作栈。
3. 动态链接：指向运行时常量池的方法引用。
4. 方法返回地址：PC 寄存器的地址。

什么是线程私有？

由于JVM的多线程是通过线程轮流切换并分配处理器执行时间的方式来实现，因此在任何一个确定的时刻，一个处理器(多核处理器则指的是一个内核)都只会执行一条线程中的指令。因此为了切换线程后能恢复到正确的执行位置，每条线程都需要独立的程序计数器，各条线程之间计数器互不影响，独立存储。我们就把类似这类区域称之为"线程私有"的内存。

3.3 本地方法栈（线程私有）

本地方法栈和虚拟机栈类似，只不过 Java 虚拟机栈是给 JVM 使用的，而本地方法栈是给本地方法使用的。

3.4 程序计数器（线程私有）

程序计数器的作用：用来记录当前线程执行的行号的。

程序计数器是一块比较小的内存空间，可以看做是当前线程所执行的字节码的行号指示器。

如果当前线程正在执行的是一个Java方法，这个计数器记录的是正在执行的虚拟机字节码指令的地址；如果正在执行的是一个Native方法，这个计数器值为空。

程序计数器内存区域是唯一一个在JVM规范中没有规定任何OOM情况的区域！

3.5 方法区（线程共享）

方法区的作用：用来存储被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等数据的。

在《Java虚拟机规范》中把此区域称之为“方法区”，而在 HotSpot 虚拟机的实现中，在 JDK 7 时此区域叫做永久代（PermGen），JDK 8 中叫做元空间（Metaspace）。

PS：永久代（PermGen）和元空间（Metaspace）是 HotSpot 中对《Java虚拟机规范》中方法区的实现，它们三者之间的关系就好比，对于一辆汽车来说它定义了一个部分叫做“动能提供装置”，但对于不同的汽车有不同的实现技术，比如对于燃油车来说，它的“动能提供装置”的实现技术就是汽油发动机（简称发动机），而对于电动汽车来说，它的“动能提供装置”的实现就是电动发动机（简称电机），发动机和电机就相当于永久代和元空间一样，它是对于“制动器”也就是方法区定义的实现。

JDK 1.8 元空间的变化

1. 对于 HotSpot 来说，JDK 8 元空间的内存属于本地内存，这样元空间的大小就不在受 JVM 最大内存的参数影响了，而是与本地内存的大小有关。
2. JDK 8 中将字符串常量池移动到了堆中。

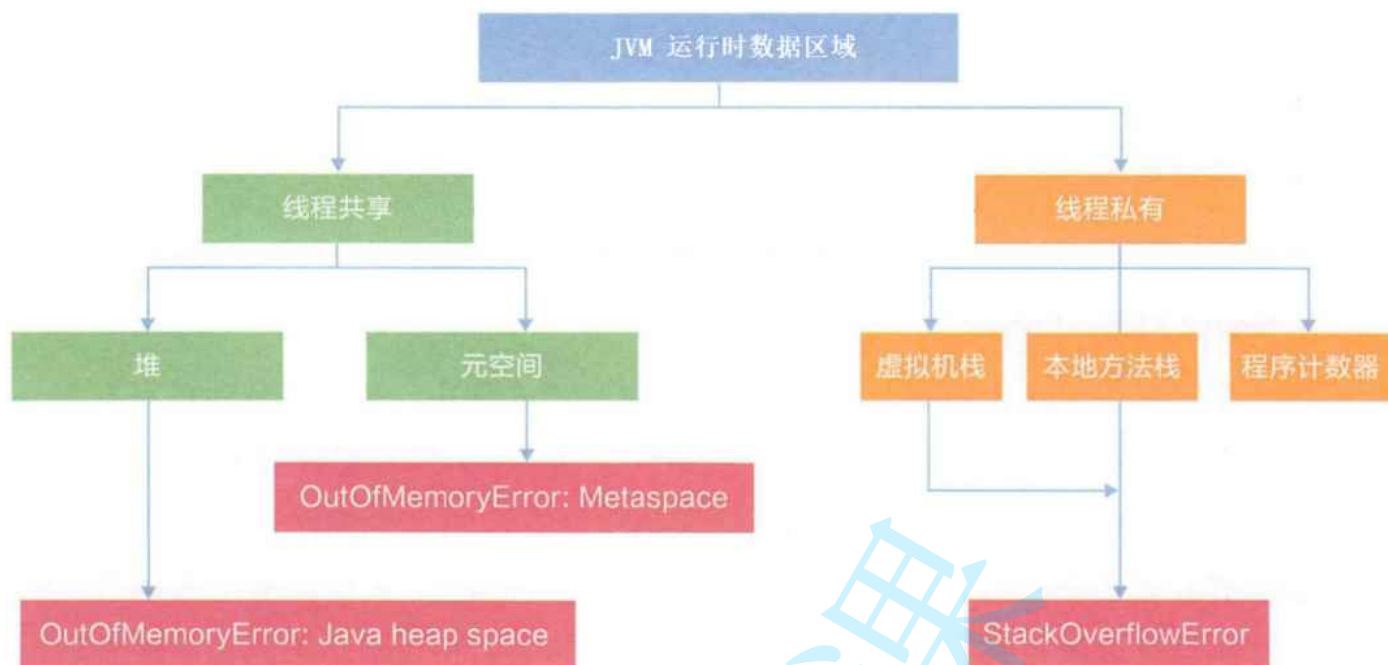
运行时常量池

运行时常量池是方法区的一部分，存放字面量与符号引用。

字面量：字符串(JDK 8 移动到堆中)、final常量、基本数据类型的值。

符号引用：类和结构的完全限定名、字段的名称和描述符、方法的名称和描述符。

小结

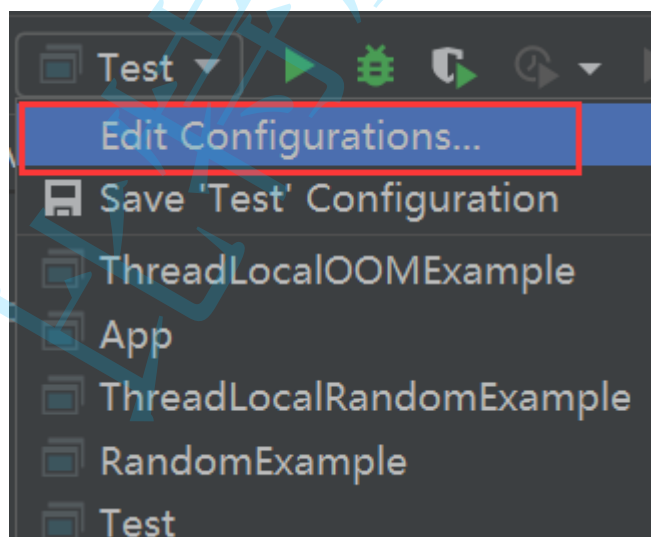


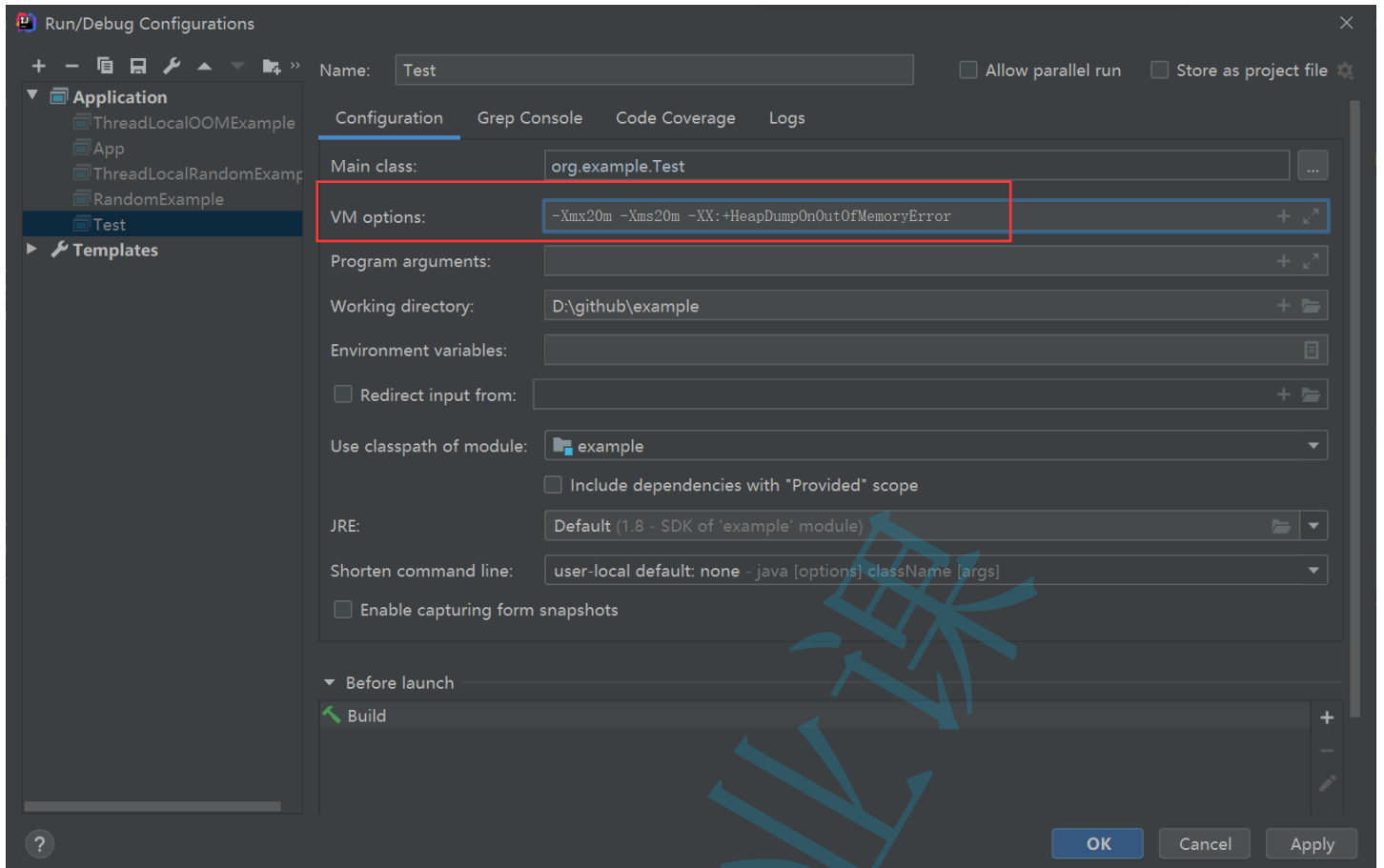
3.6 内存布局中的异常问题

① Java堆溢出

Java堆用于存储对象实例，只要不断的创建对象，并且保证GC Roots到对象之间有可达路径来避免来GC清除这些对象，那么在对象数量达到最大堆容量后就会产生内存溢出异常。

上节中已经讲到了，可以设置JVM参数-Xms:设置堆的最小值、-Xmx:设置堆最大值。下面我们来看一个Java堆OOM的测试，测试以下代码之前先设置Idea的启动参数，如下图所示：





PS: JVM 参数为:-Xmx20m -Xms20m -XX:+HeapDumpOnOutOfMemoryError

范例：观察 Java Heap OOM

```
1 /**
2  * JVM 参数为:-Xmx20m -Xms20m -XX:+HeapDumpOnOutOfMemoryError
3  * @author 38134
4  *
5  */
6 public class Test {
7     static class OOMObject {
8
9     }
10    public static void main(String[] args) {
11        List<OOMObject> list =
12            new ArrayList<>();
13        while(true) {
14            list.add(new OOMObject());
15        }
16    }
17 }
```

Java堆内存的OOM异常是实际应用中最常见的内存溢出情况。当出现Java堆内存溢出时，异常堆栈信息"java.lang.OutOfMemoryError"会进一步提示"Java heap space"。当出现"Java heap space"则很

明确的告知我们，OOM发生在堆上。

此时要对Dump出来的文件进行分析，以MAT为例。分析问题的产生到底是出现了内存泄漏(Memory Leak)还是内存溢出(Memory Overflow)

内存泄漏：泄漏对象无法被GC

内存溢出：内存对象确实还应该存活。此时要根据JVM堆参数与物理内存相比较检查是否还应该把JVM堆内存调大；或者检查对象的生命周期是否过长。

以上是我们处理Java堆内存的简单方法，处理具体这类问题需要的工具以及知识我们放到下面第四小节具体来说。

② 虚拟机栈和本地方法栈溢出

由于我们HotSpot虚拟机将虚拟机栈与本地方法栈合二为一，因此对于HotSpot来说，栈容量只需要由-Xss参数来设置。

关于虚拟机栈会产生的两种异常：

- 如果线程请求的栈深度大于虚拟机所允许的最大深度，会抛出StackOverFlow异常
- 如果虚拟机在拓展栈时无法申请到足够的内存空间，则会抛出OOM异常

范例:观察StackOverFlow异常(单线程环境下)

```
1  /**
2   * JVM参数为:-Xss128k
3   * @author 38134
4   *
5   */
6  public class Test {
7      private int stackLength = 1;
8      public void stackLeak() {
9          stackLength++;
10         stackLeak();
11     }
12     public static void main(String[] args) {
13         Test test = new Test();
14         try {
15             test.stackLeak();
16         } catch (Throwable e) {
17             System.out.println("Stack Length: "+test.stackLength);
18             throw e;
19         }
20     }
21 }
```

出现StackOverflowError异常时有错误堆栈可以阅读，比较好找到问题所在。如果使用虚拟机默认参数，栈深度在大多数情况下达到1000-2000完全没问题，对于正常的方法调用（包括递归），完全够用。

如果是因为多线程导致的内存溢出问题，在不能减少线程数的情况下，只能减少最大堆和减少栈容量的方式来换取更多线程。

范例：观察多线程下的内存溢出异常

```
1  /**
2   * JVM参数为:-Xss2M
3   * @author 38134
4   *
5   */
6  public class Test {
7
8      private void dontStop() {
9          while(true) {
10
11          }
12      }
13      public void stackLeakByThread() {
14          while(true) {
15              Thread thread = new Thread(new Runnable() {
16                  @Override
17                  public void run() {
18                      dontStop();
19                  }
20              });
21              thread.start();
22          }
23      }
24
25      public static void main(String[] args) {
26          Test test = new Test();
27          test.stackLeakByThread();
28      }
29  }
```

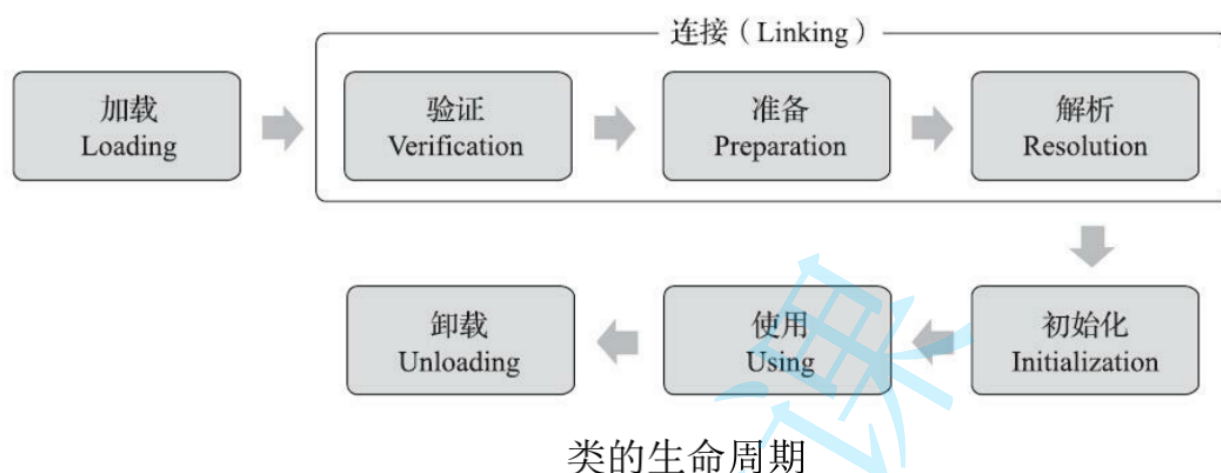
以上代码运行需谨慎。先记得保存手头所有工作。

4.JVM 类加载

① 类加载过程

从上面的图片我们可以看出整个 JVM 执行的流程中，和程序员关系最密切的就是类加载的过程了，所以接下来我们来看下类加载的执行流程。

对于一个类来说，它的生命周期是这样的：



其中前 5 步是固定的顺序并且也是类加载的过程，其中中间的 3 步我们都属于连接，所以对于类加载来说总共分为以下几个步骤：

1. 加载
2. 连接
 - a. 验证
 - b. 准备
 - c. 解析
3. 初始化

下面我们分别来看每个步骤的具体执行内容。

1) 加载

“加载”（Loading）阶段是整个“类加载”（Class Loading）过程中的一个阶段，它和类加载 Class Loading 是不同的，一个是加载 Loading 另一个是类加载 Class Loading，所以不要把二者搞混了。

在加载 Loading 阶段，Java 虚拟机需要完成以下三件事情：

- 1) 通过一个类的全限定名来获取定义此类的二进制字节流。
- 2) 将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构。
- 3) 在内存中生成一个代表这个类的 `java.lang.Class` 对象，作为方法区这个类的各种数据的访问入口。

2) 验证

验证是连接阶段的第一步，这一阶段的目的是确保Class文件的字节流中包含的信息符合《Java虚拟机规范》的全部约束要求，保证这些信息被当作代码运行后不会危害虚拟机自身的安全。

验证选项：

- 文件格式验证
- 字节码验证
- 符号引用验证...

3) 准备

准备阶段是正式为类中定义的变量（即静态变量，被static修饰的变量）分配内存并设置类变量初始值的阶段。

比如此时有这样一行代码：

```
| public static int value = 123;
```

它是初始化 value 的 int 值为 0，而非 123。

4) 解析

解析阶段是 Java 虚拟机将常量池内的符号引用替换为直接引用的过程，也就是初始化常量的过程。

5) 初始化

初始化阶段，Java 虚拟机真正开始执行类中编写的 Java 程序代码，将主导权移交给应用程序。初始化阶段就是执行类构造器方法的过程。

② 双亲委派模型

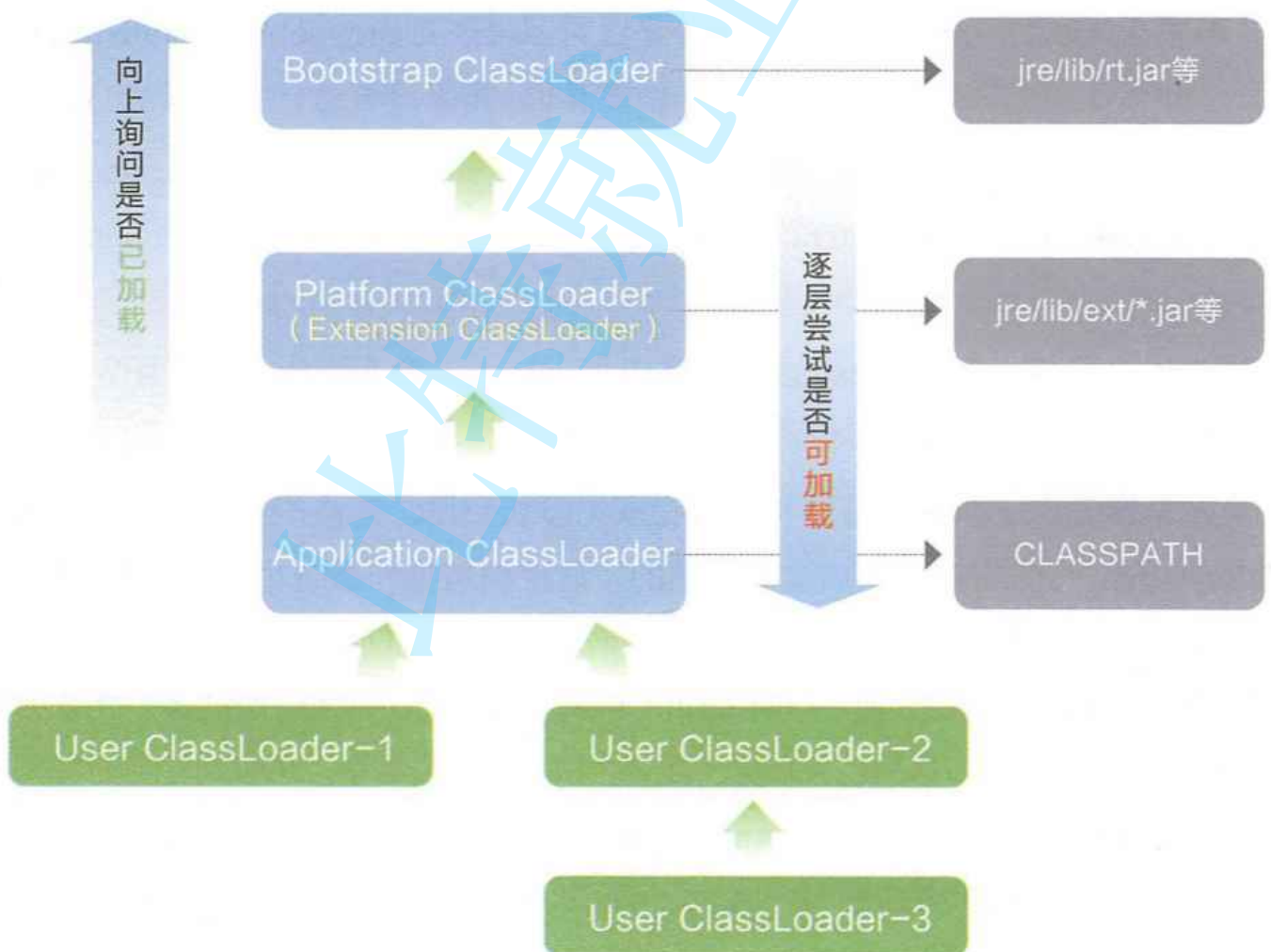
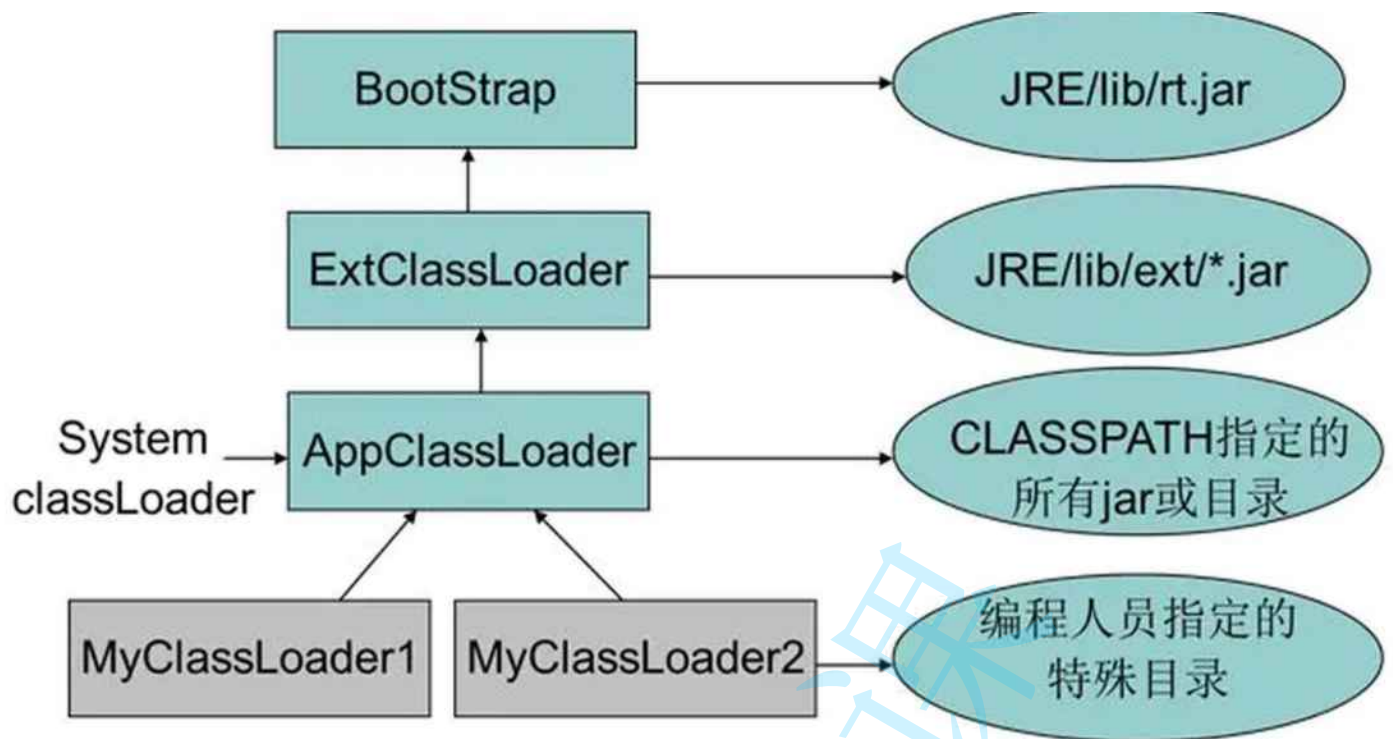
提到类加载机制，不得不提的一个概念就是“双亲委派模型”。

站在 Java 虚拟机的角度来看，只存在两种不同的类加载器：一种是启动类加载器（Bootstrap ClassLoader），这个类加载器使用 C++ 语言实现，是虚拟机自身的一部分；另外一种就是其他所有的类加载器，这些类加载器都由Java语言实现，独立存在于虚拟机外部，并且全都继承自抽象类 `java.lang.ClassLoader`。

站在 Java 开发人员的角度来看，类加载器就应当划分得更细致一些。自 JDK 1.2 以来，Java 一直保持着三层类加载器、双亲委派的类加载架构器。

什么是双亲委派模型？

如果一个类加载器收到了类加载的请求，它首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器去完成，每一个层次的类加载器都是如此，因此所有的加载请求最终都应该传送到最顶层的启动类加载器中，只有当父加载器反馈自己无法完成这个加载请求（它的搜索范围中没有找到所需的类）时，子加载器才会尝试自己去完成加载。



- 启动类加载器：加载 JDK 中 lib 目录中 Java 的核心类库，即 `$JAVA_HOME/lib` 目录。扩展类加载器。加载 `lib/ext` 目录下的类。

- 应用程序类加载器：加载我们写的应用程序。
- 自定义类加载器：根据自己的需求定制类加载器。

双亲委派模型的优点

1. 避免重复加载类：比如 A 类和 B 类都有一个父类 C 类，那么当 A 启动时就会将 C 类加载起来，那么在 B 类进行加载时就不需要在重复加载 C 类了。
2. 安全性：使用双亲委派模型也可以保证了 Java 的核心 API 不被篡改，如果没有使用双亲委派模型，而是每个类加载器加载自己的话就会出现一些问题，比如我们编写一个称为 `java.lang.Object` 类的话，那么程序运行的时候，系统就会出现多个不同的 `Object` 类，而有些 `Object` 类又是用户自己提供的因此安全性就不能得到保证了。

③ 破坏双亲委派模型

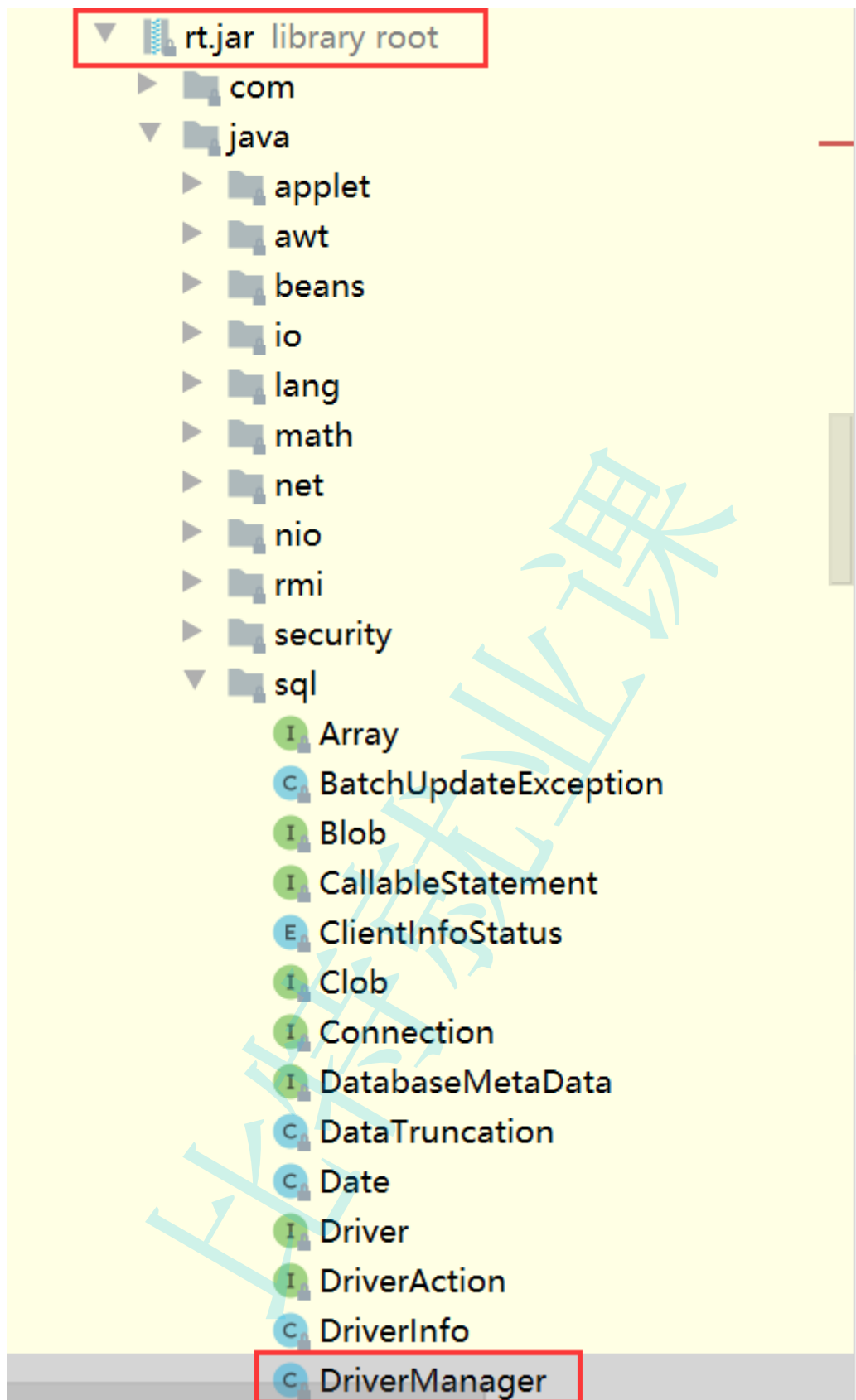
双亲委派模型虽然有其优点，但在某些情况下也存在一定的问题，比如 Java 中 SPI（Service Provider Interface，服务提供接口）机制中的 JDBC 实现。

小知识：SPI 全称 Service Provider Interface，是 Java 提供的一套用来被第三方实现或者扩展的接口，它可以用来启用框架扩展和替换组件。SPI 的作用就是为这些被扩展的 API 寻找服务实现。

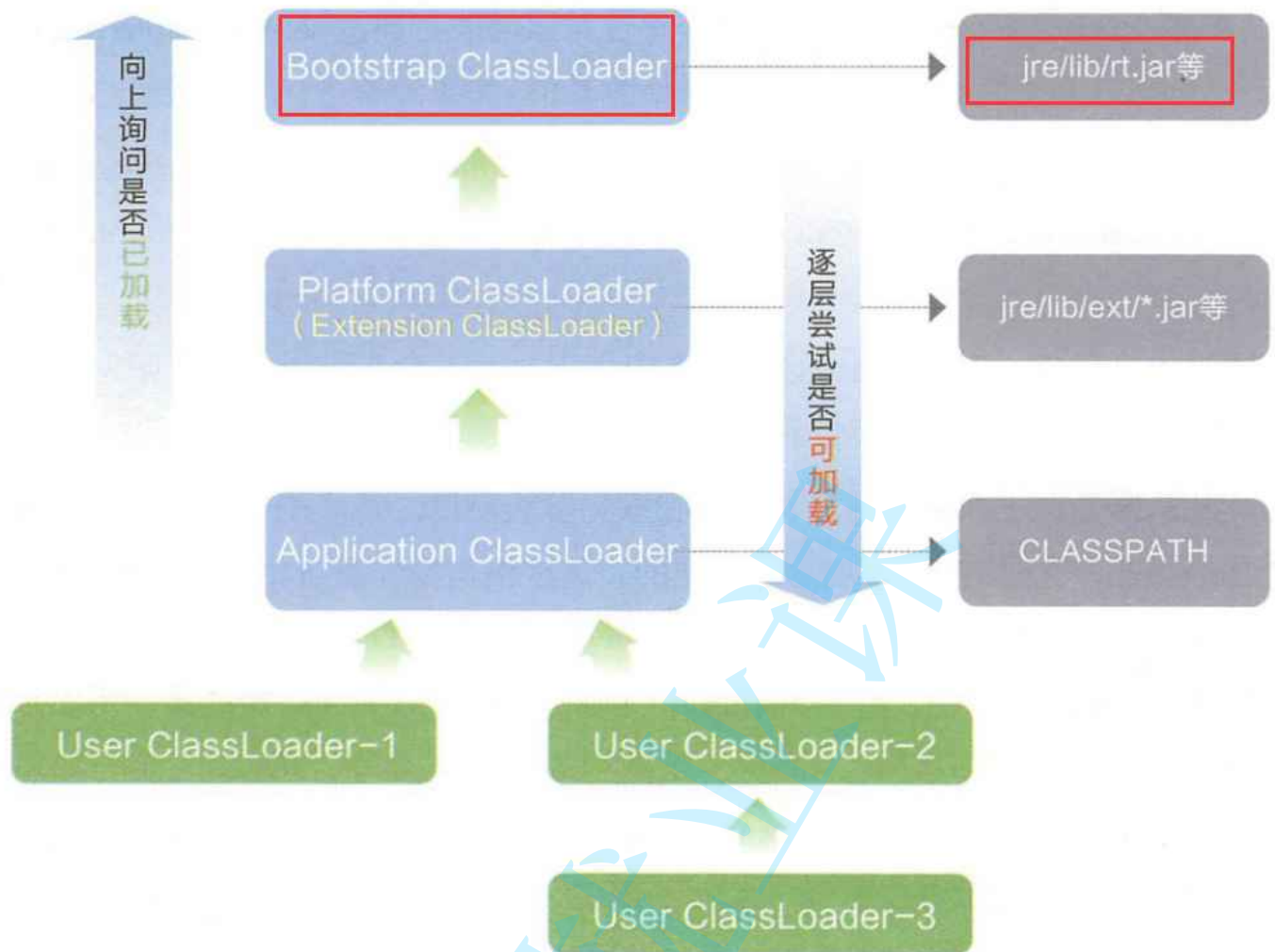
JDBC 的 Driver 接口定义在 JDK 中，其实现由各个数据库的服务商来提供，比如 MySQL 驱动包。我们先来看下 JDBC 的核心使用代码：

```
1 public class JdbcTest {
2     public static void main(String[] args){
3         Connection connection = null;
4         try {
5             connection = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306");
6         } catch (SQLException e) {
7             e.printStackTrace();
8         }
9         System.out.println(connection.getClass().getClassLoader());
10        System.out.println(Thread.currentThread().getContextClassLoader());
11        System.out.println(Connection.class.getClassLoader());
12    }
13
14 }
```

然后我们进入 `DriverManager` 的源码类就会发现它是存在系统的 `rt.jar` 中的，如下图所示：



由双亲委派模型的加载流程可知 `rt.jar` 是有顶级父类 `Bootstrap ClassLoader` 加载的，如下图所示：



而当我们进入它的 `getConnection` 源码是却发现，它在调用具体的类实现时，使用的是子类加载器（线程上下文加载器 `Thread.currentThread().getContextClassLoader`）来加载具体的数据库数据库包（如 `mysql` 的 `jar` 包），源码如下：

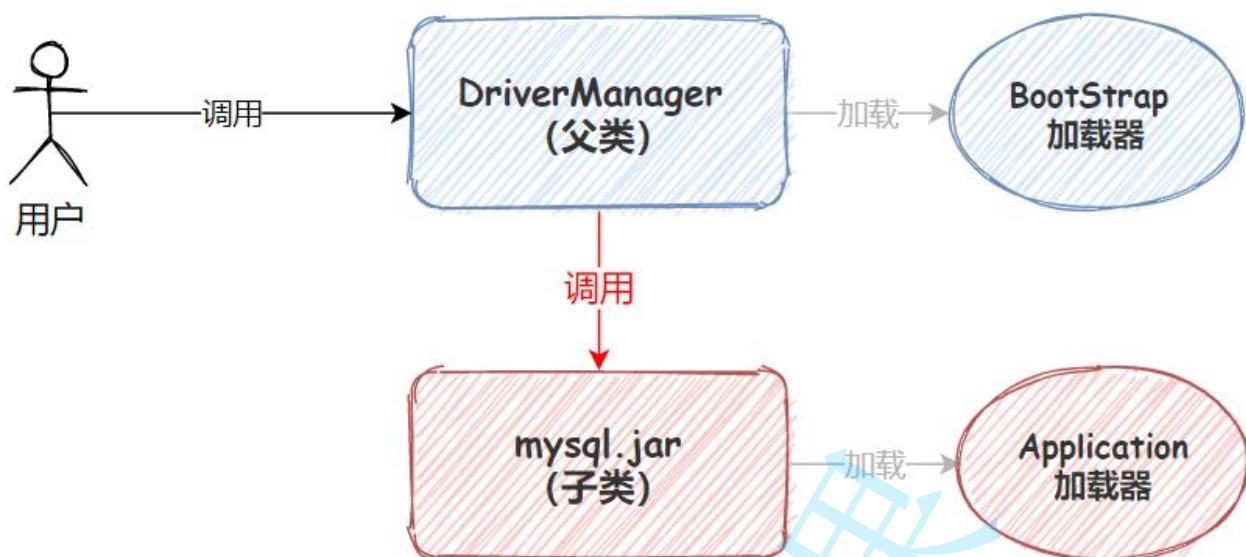
```
1 @CallerSensitive
2 public static Connection getConnection(String url,
3 java.util.Properties info) throws SQLException {
4     return (getConnection(url, info, Reflection.getCallerClass()));
5 }
6 private static Connection getConnection(
7     String url, java.util.Properties info, Class<?> caller) throws SQLException
8     ClassLoader callerCL = caller != null ? caller.getClassLoader() : null;
9     synchronized(DriverManager.class) {
10         // synchronize loading of the correct classloader.
11         if (callerCL == null) {
12             //获取线程上下文为类加载器
13             callerCL = Thread.currentThread().getContextClassLoader();
14         }
15     }
16
17     if(url == null) {
```

```

18         throw new SQLException("The url cannot be null", "08001");
19     }
20     println("DriverManager.getConnection(\"" + url + "\")");
21     SQLException reason = null;
22
23     for(DriverInfo aDriver : registeredDrivers) {
24         // isDriverAllowed 对于 mysql 连接 jar 进行加载
25         if(isDriverAllowed(aDriver.driver, callerCL)) {
26             try {
27                 println("    trying " + aDriver.driver.getClass().getName())
28                 Connection con = aDriver.driver.connect(url, info);
29                 if (con != null) {
30                     // Success!
31                     println("getConnection returning " + aDriver.driver.getC
32                         return (con);
33                 }
34             } catch (SQLException ex) {
35                 if (reason == null) {
36                     reason = ex;
37                 }
38             }
39         } else {
40             println("    skipping: " + aDriver.getClass().getName());
41         }
42     }
43
44     if (reason != null) {
45         println("getConnection failed: " + reason);
46         throw reason;
47     }
48
49     println("getConnection: no suitable driver found for " + url);
50     throw new SQLException("No suitable driver found for " + url, "08001");
51 }
52

```

这样一来就破坏了双亲委派模型，因为 DriverManager 位于 rt.jar 包，由 Bootstrap 类加载器加载，而其 Driver 接口的实现类是位于服务商提供的 Jar 包中，是由子类加载器（线程上下文加载器 Thread.currentThread().getContextClassLoader）来加载的，这样就破坏了双亲委派模型了（双亲委派模型讲的是所有类都应该交给父类来加载，但 JDBC 显然并不能这样实现）。它的交互流程图如下所示：



5.垃圾回收相关

上面讲了Java运行时内存的各个区域。对于程序计数器、虚拟机栈、本地方法栈这三部分区域而言，其生命周期与相关线程有关，随线程而生，随线程而灭。并且这三个区域的内存分配与回收具有确定性，因为当方法结束或者线程结束时，内存就自然跟着线程回收了。因此我们本节课所讲的有关内存分配和回收关注的为Java堆与方法区这两个区域。

Java堆中存放着几乎所有的对象实例，垃圾回收器在对堆进行垃圾回收前，首先要判断这些对象哪些还存活，哪些已经"死去"。判断对象是否已"死"有如下几种算法

内存 VS 对象

在Java中，所有的对象都是要存在内存中的（也可以说内存中存储的是一个对象），因此我们将内存回收，也可以叫做死亡对象的回收。

① 死亡对象的判断算法

a) 引用计数算法

引用计数描述的算法为：

给对象增加一个引用计数器，每当有一个地方引用它时，计数器就+1；当引用失效时，计数器就-1；任何时刻计数器为0的对象就是不能再被使用的，即对象已"死"。

引用计数法实现简单，判定效率也比较高，在大部分情况下都是一个不错的算法。比如Python语言就采用引用计数法进行内存管理。

但是，在主流的JVM中没有选用引用计数法来管理内存，最主要的原因就是引用计数法无法解决对象的循环引用问题

范例：观察循环引用问题

```
1  /**
2   * JVM参数 :-XX:+PrintGC
3   * @author 38134
4   *
5   */
6  public class Test {
7      public Object instance = null;
8      private static int _1MB = 1024 * 1024;
9      private byte[] bigSize = new byte[2 * _1MB];
10     public static void testGC() {
11         Test test1 = new Test();
12         Test test2 = new Test();
13         test1.instance = test2;
14         test2.instance = test1;
15         test1 = null;
16         test2 = null;
17         // 强制jvm进行垃圾回收
18         System.gc();
19     }
20     public static void main(String[] args) {
21         testGC();
22     }
23 }
```

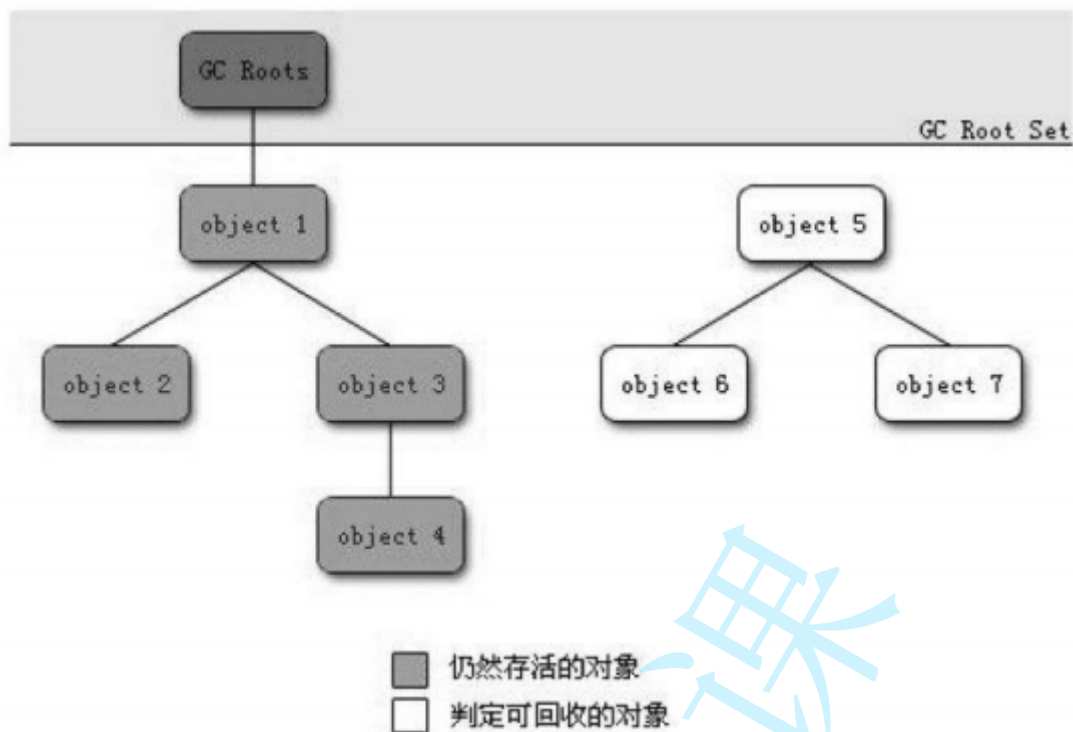
```
1 [GC (System.gc()) 6092K->856K(125952K), 0.0007504 secs]
```

从结果可以看出，GC日志包含"6092K->856K(125952K)"，意味着虚拟机并没有因为这两个对象互相引用就不回收他们。即JVM并不使用引用计数法来判断对象是否存活。

b) 可达性分析算法

在上面我们讲了，Java并不采用引用计数法来判断对象是否已"死"，而采用"可达性分析"来判断对象是否存活(同样采用此法的还有C#、Lisp-最早的一门采用动态内存分配的语言)。

此算法的核心思想为：通过一系列称为"GC Roots"的对象作为起始点，从这些节点开始向下搜索，搜索走过的路径称之为"引用链"，当一个对象到GC Roots没有任何的引用链相连时(从GC Roots到这个对象不可达)时，证明此对象是不可用的。以下图为例：



对象Object5-Object7之间虽然彼此还有关联，但是它们到GC Roots是不可达的，因此他们会被判定为可回收对象。

在Java语言中，可作为GC Roots的对象包含下面几种：

1. 虚拟机栈(栈帧中的本地变量表)中引用的对象；
2. 方法区中类静态属性引用的对象；
3. 方法区中常量引用的对象；
4. 本地方法栈中 JNI(Native方法)引用的对象。

从上面我们可以看出“引用”的功能，除了最早我们使用它（引用）来查找对象，现在我们还可以使用“引用”来判断死亡对象了。所以在 JDK1.2 时，Java 对引用的概念做了扩充，将引用分为强引用(Strong Reference)、软引用(Soft Reference)、弱引用(Weak Reference)和虚引用(Phantom Reference)四种，这四种引用的强度依次递减。

1. 强引用：强引用指的是在程序代码之中普遍存在的，类似于"Object obj = new Object()"这类的引用，只要强引用还存在，垃圾回收器永远不会回收掉被引用的对象实例。
2. 软引用：软引用是用来描述一些还有用但是不是必须的对象。对于软引用关联着的对象，在系统将要发生内存溢出之前，会把这些对象列入回收范围之中进行第二次回收。如果这次回收还是没有足够的内存，才会抛出内存溢出异常。在JDK1.2之后，提供了SoftReference类来实现软引用。
3. 弱引用：弱引用也是用来描述非必需对象的。但是它的强度要弱于软引用。被弱引用关联的对象只能生存到下一次垃圾回收发生之前。当垃圾回收器开始进行工作时，无论当前内容是否够用，都会回收掉只被弱引用关联的对象。在JDK1.2之后提供了WeakReference类来实现弱引用。

4. 虚引用：虚引用也被称为幽灵引用或者幻影引用，它是最弱的一种引用关系。一个对象是否有虚引用的存在，完全不会对其生存时间构成影响，也无法通过虚引用来取得一个对象实例。为一个对象设置虚引用的唯一目的就是能在这个对象被收集器回收时收到一个系统通知。在JDK1.2之后，提供了PhantomReference类来实现虚引用。

② 垃圾回收算法

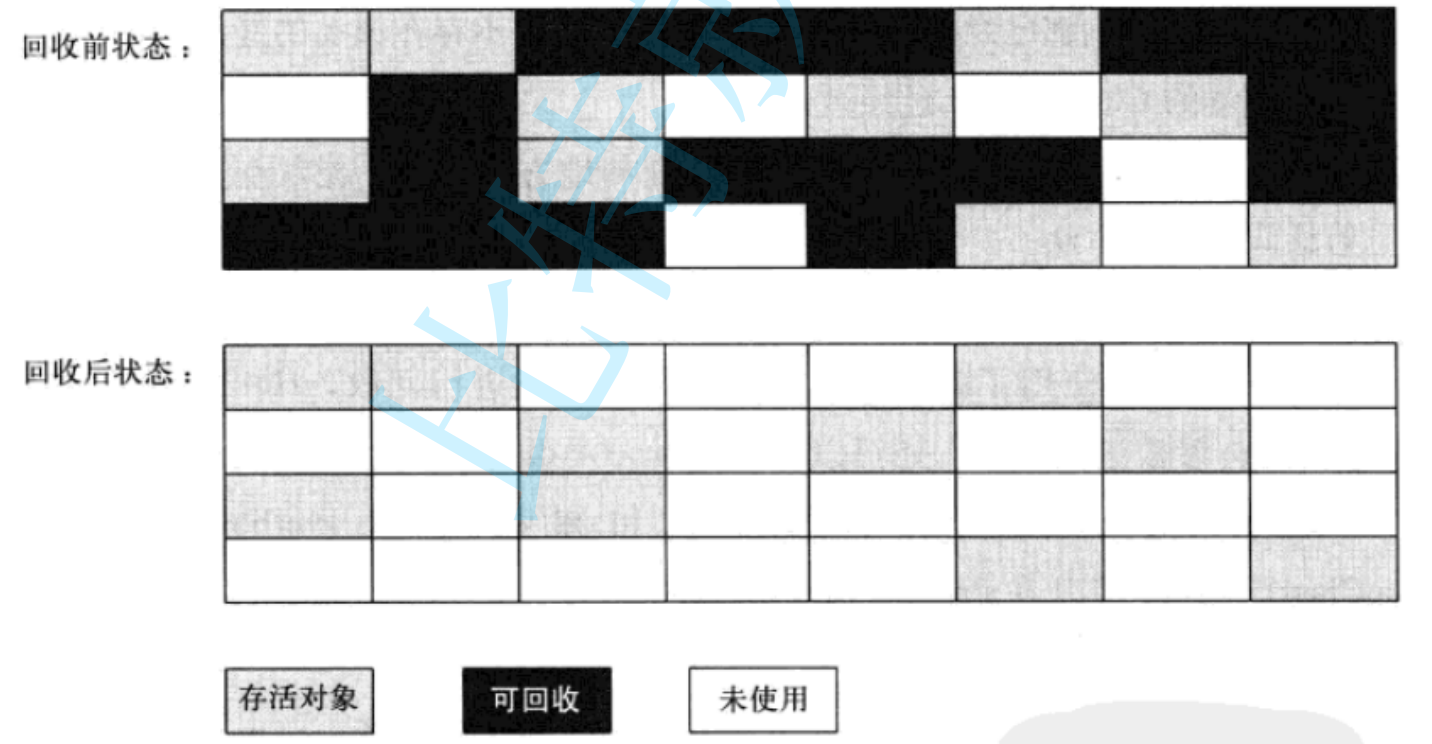
通过上面的学习我们可以将死亡对象标记出来了，标记出来之后我们就可以进行垃圾回收操作了，在正式学习垃圾收集器之前，我们先看下垃圾回收机器使用的几种算法（这些算法是垃圾收集器的指导思想）。

a) 标记-清除算法

"标记-清除"算法是最基础的收集算法。算法分为"标记"和"清除"两个阶段：首先标记出所有需要回收的对象，在标记完成后统一回收所有被标记的对象(标记过程见3.1.2章节)。后续的收集算法都是基于这种思路并对其不足加以改进而已。

"标记-清除"算法的不足主要有两个：

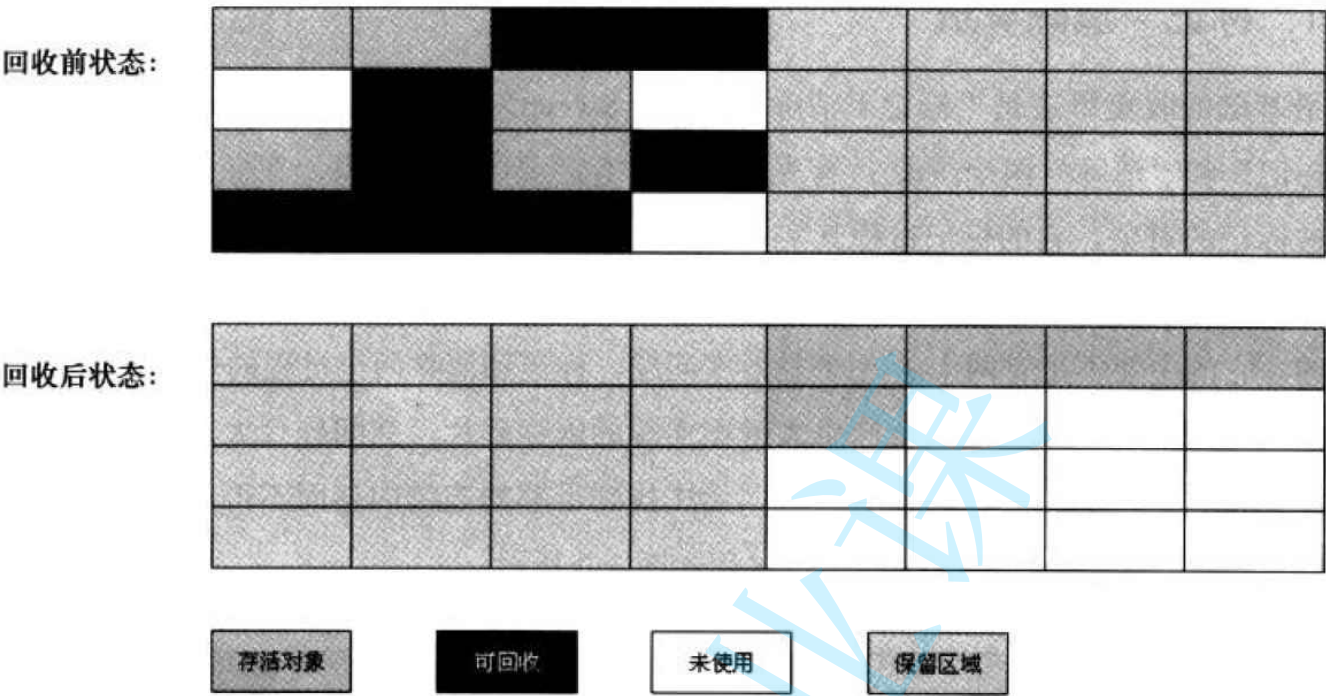
- 1. 效率问题：标记和清除这两个过程的效率都不高
- 2. 空间问题：标记清除后会产生大量不连续的内存碎片，空间碎片太多可能会导致以后在程序运行中需要分配较大对象时，无法找到足够连续内存而不得不提前触发另一次垃圾收集。



b) 复制算法

"复制"算法是为了解决"标记-清理"的效率问题。它将可用内存按容量划分为大小相等的两块，每次只使用其中的一块。当这块内存需要进行垃圾回收时，会将此区域还存活着的对象复制到另一块上面，然后再把已经使用过的内存区域一次清理掉。这样做的好处是每次都是对整个半区进行内存回收，内

存分配时也就不需要考虑内存碎片等复杂情况，只需要移动堆顶指针，按顺序分配即可。此算法实现简单，运行高效。算法的执行流程如下图：



现在的商用虚拟机(包括HotSpot都是采用这种收集算法来回收新生代)

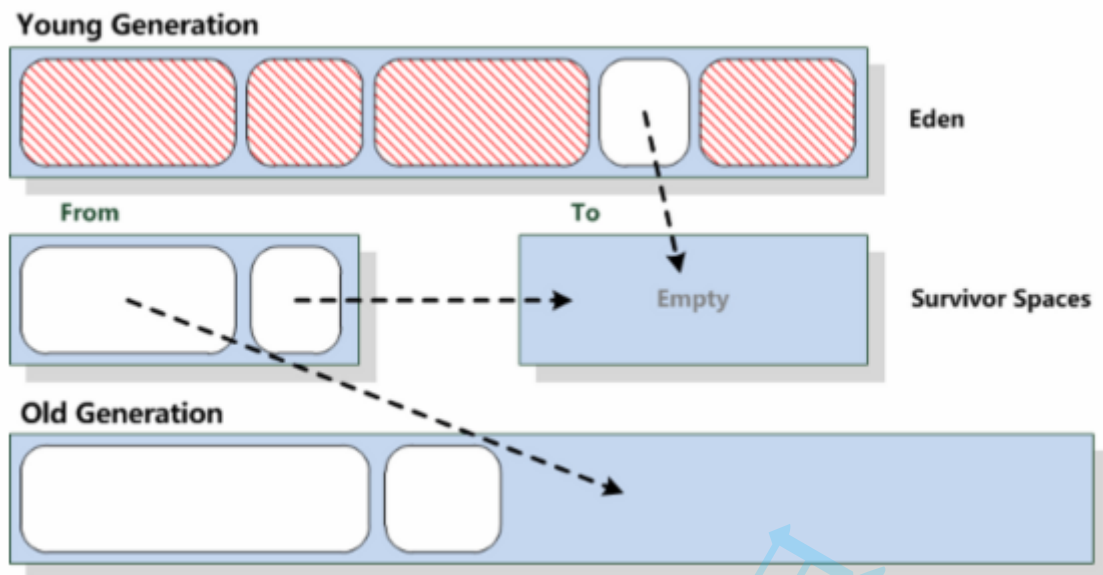
新生代中98%的对象都是"朝生夕死"的，所以并不需要按照1：1的比例来划分内存空间，而是将内存(新生代内存)分为一块较大的Eden(伊甸园)空间和两块较小的Survivor(幸存者)空间，每次使用Eden和其中一块Survivor（两个Survivor区域一个称为From区，另一个称为To区域）。当回收时，将Eden和Survivor中还存活的对象一次性复制到另一块Survivor空间上，最后清理掉Eden和刚才用过的Survivor空间。

当Survivor空间不够用时，需要依赖其他内存(老年代)进行分配担保。

HotSpot默认Eden与Survivor的大小比例是8：1，也就是说Eden:Survivor From : Survivor To = 8:1:1。所以每次新生代可用内存空间为整个新生代容量的90%,而剩下的10%用来存放回收后存活的对象。

HotSpot实现的复制算法流程如下:

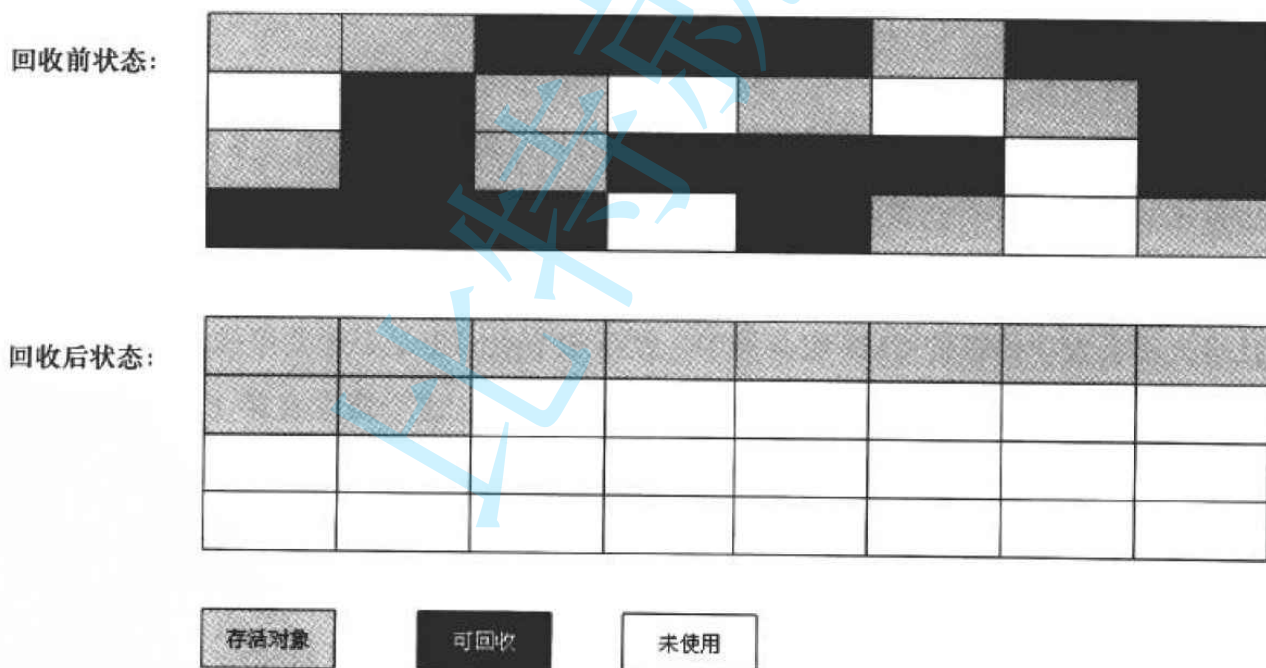
1. 当Eden区满的时候,会触发第一次Minor gc,把还活着的对象拷贝到Survivor From区；当Eden区再次触发Minor gc的时候,会扫描Eden区和From区域,对两个区域进行垃圾回收,经过这次回收后还存活的对象,则直接复制到To区域,并将Eden和From区域清空。
2. 当后续Eden又发生Minor gc的时候,会对Eden和To区域进行垃圾回收,存活的对象复制到From区域,并将Eden和To区域清空。
3. 部分对象会在From和To区域中复制来复制去,如此交换15次(由JVM参数MaxTenuringThreshold决定,这个参数默认是15),最终如果还是存活,就存入到老年代



c) 标记-整理算法

复制收集算法在对象存活率较高时会进行较多的复制操作，效率会变低。因此在老年代一般不能使用复制算法。

针对老年代的特点，提出了一种称之为"标记-整理算法"。标记过程仍与"标记-清除"过程一致，但后续步骤不是直接对可回收对象进行清理，而是让所有存活对象都向一端移动，然后直接清理掉端边界以外的内存。流程图如下：



d) 分代算法

分代算法和上面讲的 3 种算法不同，分代算法是通过区域划分，实现不同区域和不同的垃圾回收策略，从而实现更好的垃圾回收。这就好比中国的一国两制方针一样，对于不同的情况和地域设置更符合当地的规则，从而实现更好的管理，这就时分代算法的设计思想。

当前 JVM 垃圾收集都采用的是"分代收集(Generational Collection)"算法，这个算法并没有新思想，只是根据对象存活周期的不同将内存划分为几块。一般是把Java堆分为新生代和老年代。**在新生代中，每次垃圾回收都有大批对象死去，只有少量存活，因此我们采用复制算法；而老年代中对象存活率高、没有额外空间对它进行分配担保，就必须采用"标记-清理"或者"标记-整理"算法。**

哪些对象会进入新生代？哪些对象会进入老年代？

- 新生代：一般创建的对象都会进入新生代；
- 老年代：大对象和经历了 N 次（一般情况默认是 15 次）垃圾回收依然存活下来的对象会从新生代移动到老年代。

面试题：请问了解Minor GC和Full GC么，这两种GC有什么不一样吗

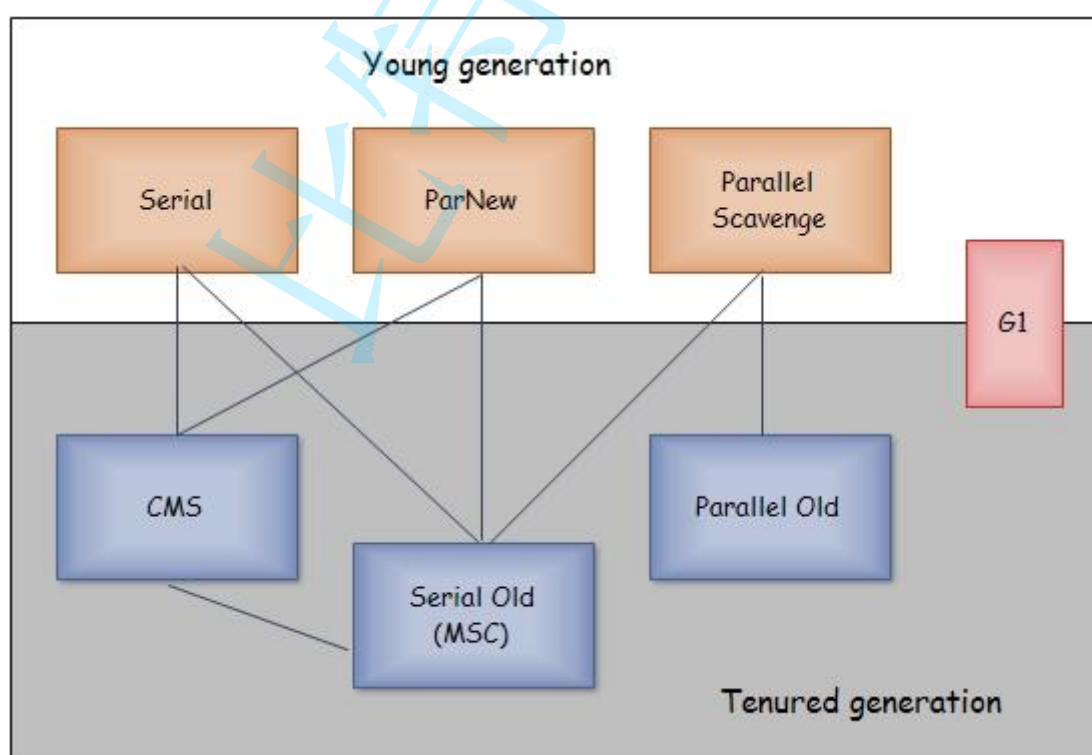
1. Minor GC又称为新生代GC：指的是发生在新生代的垃圾收集。因为Java对象大多都具备朝生夕灭的特性，因此Minor GC(采用复制算法)非常频繁，一般回收速度也比较快。
2. Full GC 又称为 老年代GC或者Major GC：指发生在老年代的垃圾收集。出现了Major GC，经常会伴随至少一次的Minor GC(并非绝对，在Parallel Scavenge收集器中就有直接进行Full GC的策略选择过程)。Major GC的速度一般会比Minor GC慢10倍以上。

③ 垃圾收集器

如果说上面我们讲的收集算法是内存回收的方法论，那么垃圾收集器就是内存回收的具体实现。

垃圾收集器的作用：垃圾收集器是为了保证程序能够正常、持久运行的一种技术，它是将程序中不用的死亡对象也就是垃圾对象进行清除，从而保证了新对象能够正常申请到内存空间。

以下这些收集器是 HotSpot 虚拟机随着不同版本推出的重要的垃圾收集器：



上图展示了7种作用于不同分代的收集器，如果两个收集器之间存在连线，就说明他们之间可以搭配使用。所处的区域，表示它是属于新生代收集器还是老年代收集器。在讲具体的收集器之前我们先来明确三个概念：

- 并行(Parallel) : 指多条垃圾收集线程并行工作，用户线程仍处于等待状态
- 并发(Concurrent) : 指用户线程与垃圾收集线程同时执行(不一定并行，可能会交替执行)，用户程序继续运行，而垃圾收集程序在另外一个CPU上。
- 吞吐量:就是CPU用于运行用户代码的时间与CPU总消耗时间的比值。

$$\text{吞吐量} = \frac{\text{运行用户代码时间}}{\text{运行用户代码时间} + \text{垃圾收集时间}}$$

例如：虚拟机总共运行了100分钟，其中垃圾收集花掉1分钟，那吞吐量就是99%。

为什么会有这么多垃圾收集器？

自从有了 Java 语言就有了垃圾收集器，这么多垃圾收集器其实是历史发展的产物。最早的垃圾收集器为 Serial，也就是串行执行的垃圾收集器，Serial Old 为串行的老年代收集器，而随着时间的发展，为了提升更高的性能，于是有了 Serial 多线程版的垃圾收集器 ParNew。后来人们想要更高吞吐量的垃圾收集器，吞吐量是指单位时间内成功回收垃圾的数量，于是就有了吞吐量优先的垃圾收集器 Parallel Scavenge（吞吐量优先的新生代垃圾收集器）和 Parallel Old（吞吐量优先的老年代垃圾收集器）。随着技术的发展后来又有了 CMS（Concurrent Mark Sweep）垃圾收集器，CMS 可以兼顾吞吐量和以获取最短回收停顿时间为目标的收集器，在 JDK 1.8（包含）之前 BS 系统的主流垃圾收集器，而在 JDK 1.8 之后，出现了第一个既不完全属于新生代也不完全属于老年代的垃圾收集器 G1（Garbage First），G1 提供了基本不需要停止程序就可以收集垃圾的技术，下面我们来看每种垃圾收集器具体的介绍。

a) Serial收集器(新生代收集器,串行GC)【选学】

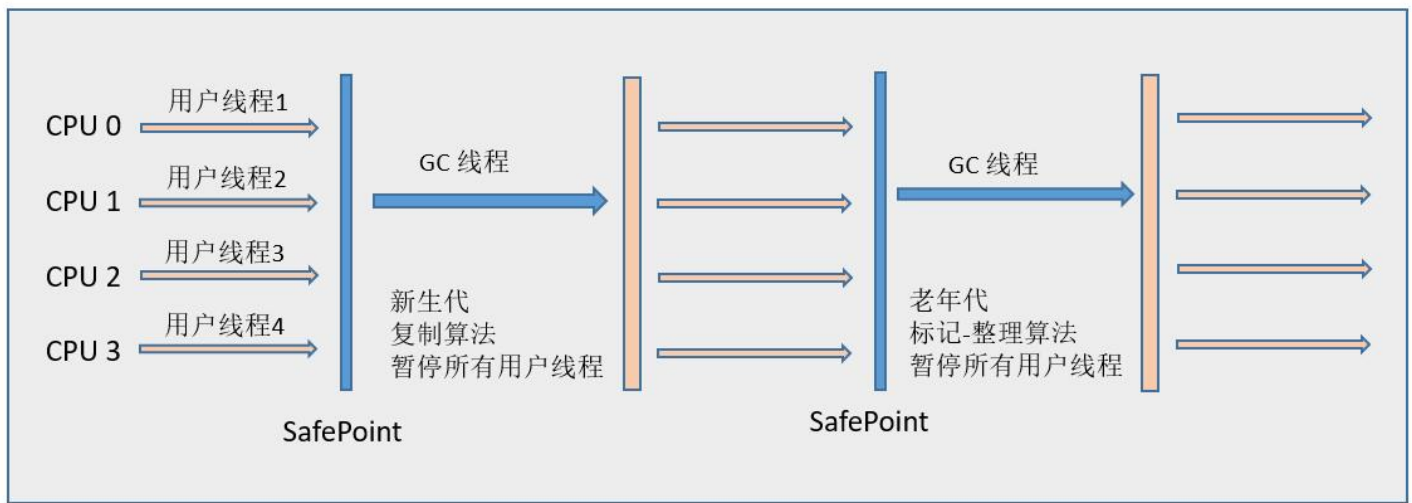
Serial 收集器是最基本、发展历史最悠久的收集器，曾经（在JDK 1.3.1之前）是虚拟机新生代收集的唯一选择。

- **特性：**
- 这个收集器是一个单线程的收集器，但它的“单线程”的意义并不仅仅说明它只会使用一个CPU或一条收集线程去完成垃圾收集工作，更重要的是在它进行垃圾收集时，必须暂停其他所有的工作线程，直到它收集结束(Stop The World，译为停止整个程序，简称 STW)。
- **应用场景：**

Serial收集器是虚拟机运行在Client模式下的默认新生代收集器。

- **优势：**

简单而高效（与其他收集器的单线程比），对于限定单个CPU的环境来说，Serial收集器由于没有线程交互的开销，专心做垃圾收集自然可以获得最高的单线程收集效率。实际上到现在为止：它依然是虚拟机运行在Client模式下的默认新生代收集器



b) ParNew收集器(新生代收集器,并行GC)【选学】

ParNew收集器其实就是Serial收集器的多线程版本，除了使用多条线程进行垃圾收集之外，其余行为包括Serial收集器可用的所有控制参数、收集算法、Stop The World、对象分配规则、回收策略等都与Serial收集器完全一样，在实现上，这两种收集器也共用了相当多的代码。

- **特性：**

Serial收集器的多线程版本

- **应用场景：**

ParNew收集器是许多运行在Server模式下的虚拟机中首选的新生代收集器。

作为Server的首选收集器之中有一个与性能无关的很重要的原因是：除了Serial收集器外，目前只有它能与CMS收集器配合工作。

在JDK 1.5时期，HotSpot推出了一款在强交互应用中几乎可认为有划时代意义的垃圾收集器——CMS收集器，这款收集器是HotSpot虚拟机中第一款真正意义上的并发收集器，它第一次实现了让垃圾收集线程与用户线程同时工作。

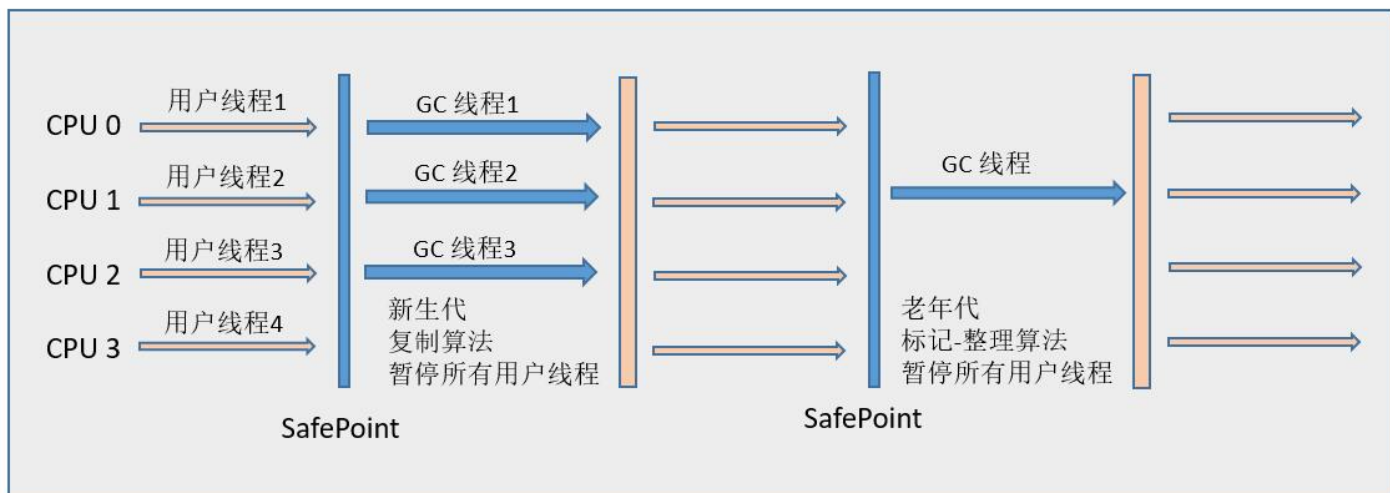
不幸的是，CMS作为老年代的收集器，却无法与JDK 1.4.0中已经存在的新生代收集器Parallel Scavenge配合工作，所以在JDK 1.5中使用CMS来收集老年代的时候，新生代只能选择ParNew或者Serial收集器中的一个。

- **对比分析：**

与Serial收集器对比：

ParNew收集器在单CPU的环境中绝对不会有比Serial收集器更好的效果，甚至由于存在线程交互的开销，该收集器在通过超线程技术实现的两个CPU的环境中都不能百分之百地保证可以超越Serial收集器。

然而，随着可以使用的CPU的数量的增加，它对于GC时系统资源的有效利用还是很有好处的。



c) Parallel Scavenge收集器(新生代收集器,并行GC)【选学】

- 特性:

Parallel Scavenge收集器是一个**新生代收集器**，它也是使用**复制算法**的收集器，又是**并行**的多线程收集器。

Parallel Scavenge收集器使用两个参数控制吞吐量：

- 1 XX:MaxGCPauseMillis 控制最大的垃圾收集停顿时间
- 2 XX:GCRatio 直接设置吞吐量的大小

直观上，只要最大的垃圾收集停顿时间越小，吞吐量是越高的，但是**GC停顿时间的缩短是以牺牲吞吐量和新生代空间作为代价的。**比如原来10秒收集一次，每次停顿100毫秒，现在变成5秒收集一次，每次停顿70毫秒。停顿时间下降的同时，吞吐量也下降了。

- 应用场景:

停顿时间越短就越适合需要与用户交互的程序，良好的响应速度能提升用户体验，而高吞吐量则可以高效率地利用CPU时间，尽快完成程序的运算任务，主要适合在后台运算而不需要太多交互的任务。

- 对比分析:

- **Parallel Scavenge收集器 VS CMS等收集器:**

Parallel Scavenge收集器的特点是它的关注点与其他收集器不同，CMS等收集器的关注点是尽可能地缩短垃圾收集时用户线程的停顿时间，而Parallel Scavenge收集器的目标则是达到一个**可控制的吞吐量**（Throughput）。

由于与吞吐量关系密切，Parallel Scavenge收集器也经常称为“吞吐量优先”收集器。

- **Parallel Scavenge收集器 VS ParNew收集器:**

Parallel Scavenge收集器与ParNew收集器的一个重要区别是它具有自适应调节策略。

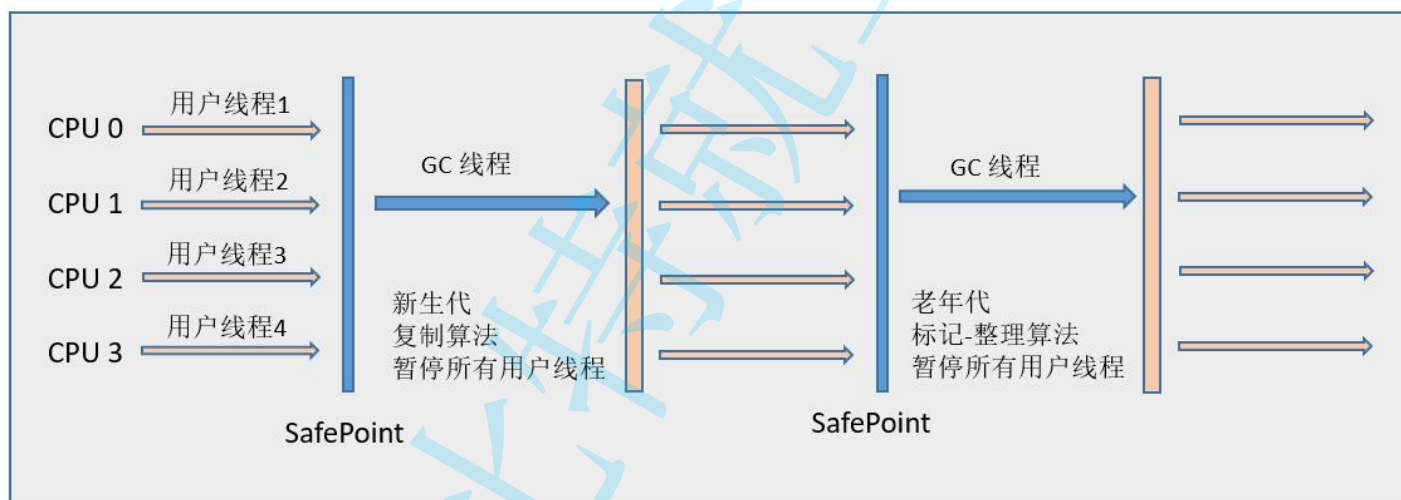
- GC自适应的调节策略:

- Parallel Scavenge收集器有一个参数- `XX:+UseAdaptiveSizePolicy`。当这个参数打开之后，就不需要手工指定新生代的大小、Eden与Survivor区的比例、晋升老年代对象年龄等细节参数了，虚拟机会根据当前系统的运行情况收集性能监控信息，动态调整这些参数以提供最合适的停顿时间或者最大的吞吐量，这种调节方式称为GC自适应的调节策略（GC Ergonomics）。

注：工作流程图同ParNew

d) Serial Old收集器(老年代收集器，串行GC)【选学】

- 特性：**
Serial Old是Serial收集器的**老年代版本**，它同样是一个**单线程收集器**，使用**标记—整理**算法。
- 应用场景：**
 - Client模式**
Serial Old收集器的主要意义也是在于给Client模式下的虚拟机使用。
 - Server模式**
如果在Server模式下，那么它主要还有两大用途：一种用途是在JDK 1.5以及之前的版本中与Parallel Scavenge收集器搭配使用，另一种用途就是作为CMS收集器的后备预案，在并发收集发生Concurrent Mode Failure时使用。



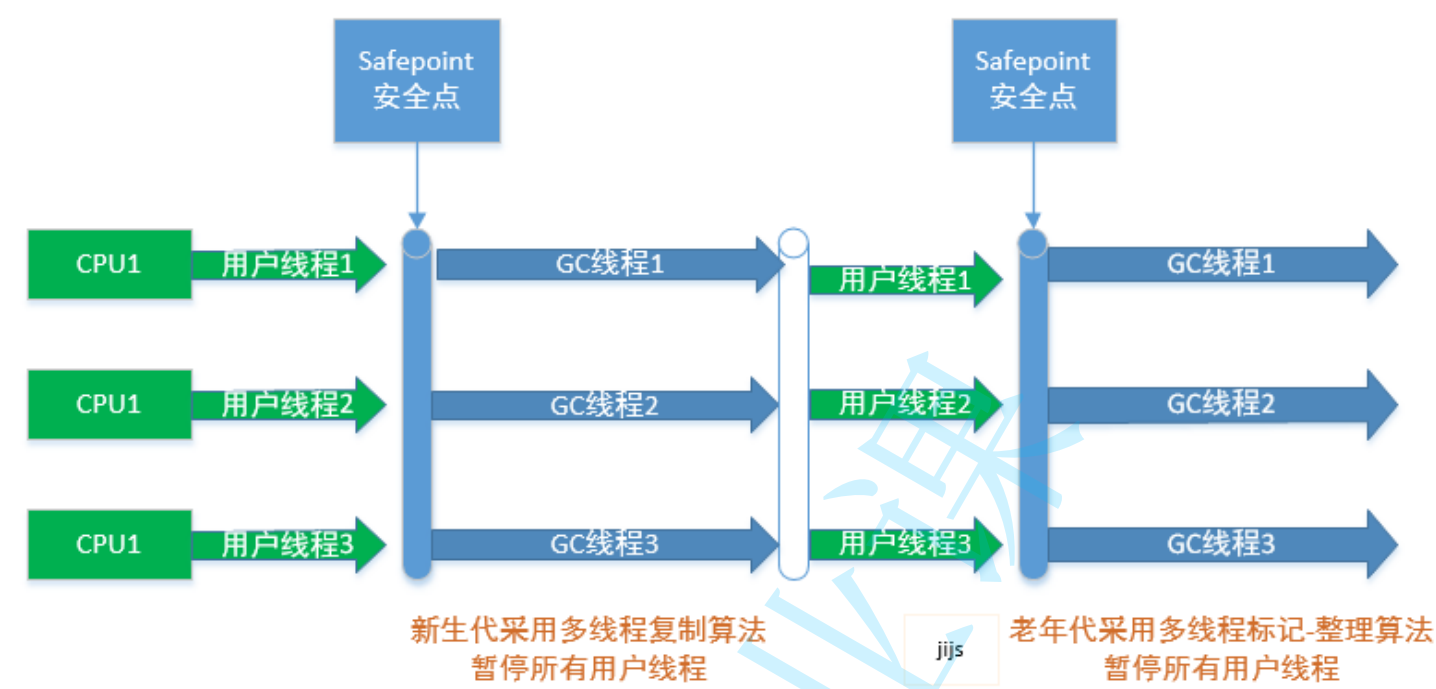
e) Parallel Old收集器(老年代收集器，并行GC)【选学】

- 特性：**
Parallel Old是Parallel Scavenge收集器的**老年代版本**，使用**多线程**和**“标记—整理”**算法。
- 应用场景：**

在注重吞吐量以及CPU资源敏感的场所，都可以优先考虑Parallel Scavenge加Parallel Old收集器。

这个收集器是在JDK 1.6中才开始提供的，在此之前，新生代Parallel Scavenge收集器一直处于比较尴尬的状态。原因是，如果新生代选择了Parallel Scavenge收集器，老年代除了Serial Old收集器外别无选择（Parallel Scavenge收集器无法与CMS收集器配合工作）。由于老年代Serial Old收集器在服务端应用性能上的“拖累”，使用了Parallel Scavenge收集器也未必能在整体应用上获得吞吐量最大化。

的效果，由于单线程的老年代收集中无法充分利用服务器多CPU的处理能力，在老年代很大而且硬件比较高级的环境中，这种组合的吞吐量甚至还不一定有ParNew加CMS的组合“给力”。直到Parallel Old收集器出现后，“吞吐量优先”收集器终于有了比较名副其实的应用组合。



f) CMS收集器（老年代收集器，并发GC）

- 特性：
CMS（Concurrent Mark Sweep）收集器是一种以获取最短回收停顿时间为目标的收集器。目前很大一部分的Java应用集中在互联网站或者B/S系统的服务端上，这类应用尤其重视服务的响应速度，希望系统停顿时间最短，以给用户带来较好的体验。CMS收集器就非常符合这类应用的需求。
- CMS收集器是基于“标记—清除”算法实现的，它的运作过程相对于前面几种收集器来说更复杂一些，整个过程分为4个步骤：
 - 初始标记（CMS initial mark）
初始标记仅仅只是标记一下GC Roots能直接关联到的对象，速度很快，需要“Stop The World”。
 - 并发标记（CMS concurrent mark）
并发标记阶段就是进行GC Roots Tracing的过程。
 - 重新标记（CMS remark）
重新标记阶段是为了修正并发标记期间因用户程序继续运作而导致标记产生变动的那一部分对象的标记记录，这个阶段的停顿时间一般会比初始标记阶段稍长一些，但远比并发标记的时间短，仍然需要“Stop The World”。
 - 并发清除（CMS concurrent sweep）
并发清除阶段会清除对象。

由于整个过程中耗时最长的并发标记和并发清除过程收集器线程都可以与用户线程一起工作，所以，从总体上来说，CMS收集器的内存回收过程是与用户线程一起并发执行的。

- **优点：**

CMS是一款优秀的收集器，它的主要优点在名字上已经体现出来了：**并发收集、低停顿。**

- **缺点：**

- CMS收集器对CPU资源非常敏感

其实，面向并发设计的程序都对CPU资源比较敏感。在并发阶段，它虽然不会导致用户线程停顿，但是会因为占用了一部分线程（或者说CPU资源）而导致应用程序变慢，总吞吐量会降低。

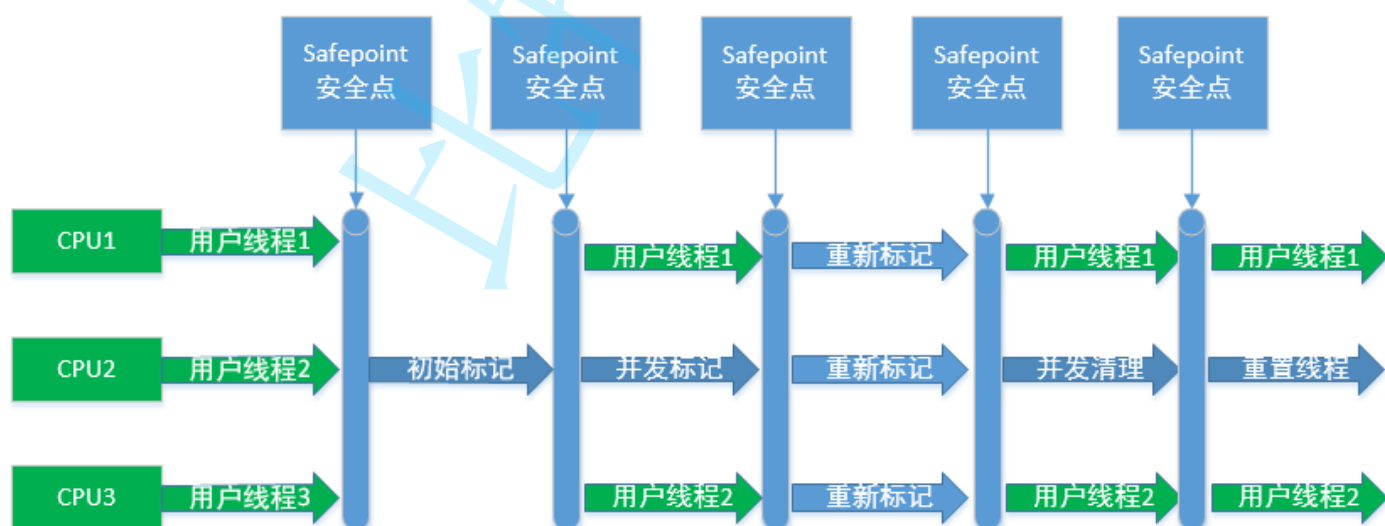
CMS默认启动的回收线程数是 $(\text{CPU数量} + 3) / 4$ ，也就是当CPU在4个以上时，并发回收时垃圾收集线程不少于25%的CPU资源，并且随着CPU数量的增加而下降。但是当CPU不足4个（譬如2个）时，CMS对用户程序的影响就可能变得很大。

- CMS收集器无法处理浮动垃圾

CMS收集器无法处理浮动垃圾，可能出现“Concurrent Mode Failure”失败而导致另一次Full GC的产生。由于CMS并发清理阶段用户线程还在运行着，伴随程序运行自然就还会有新的垃圾不断产生，这一部分垃圾出现在标记过程之后，CMS无法在当次收集中处理掉它们，只好留待下一次GC时再清理掉。这一部分垃圾就称为“浮动垃圾”。也是由于在垃圾收集阶段用户线程还需要运行，那也就还需要预留有足够的内存空间给用户线程使用，因此CMS收集器不能像其他收集器那样等到老年代几乎完全被填满了再进行收集，需要预留一部分空间提供并发收集时的程序运作使用。要是CMS运行期间预留的内存无法满足程序需要，就会出现一次“Concurrent Mode Failure”失败，这时虚拟机将启动后备预案：临时启用Serial Old收集器来重新进行老年代的垃圾收集，这样停顿时间就很长了。

- CMS收集器会产生大量空间碎片

CMS是一款基于“标记—清除”算法实现的收集器，这意味着收集结束时会有大量空间碎片产生。空间碎片过多时，将会给大对象分配带来很大麻烦，往往会出现老年代还有很大空间剩余，但是无法找到足够大的连续空间来分配当前对象，不得不提前触发一次Full GC。



新生代采用多线程标记-清除算法
初始标记和重新标记需要STW

jjjs

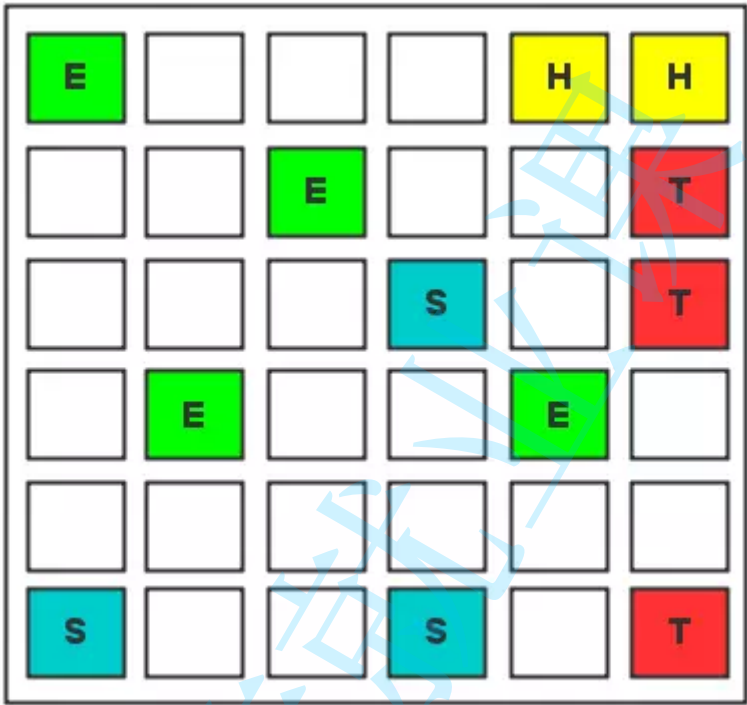
g) G1收集器(唯一一款全区域的垃圾回收器)

G1（Garbage First）垃圾回收器是用在heap memory很大的情况下，把heap划分为很多很多的region块，然后并行的对其进行垃圾回收。

G1垃圾回收器在清除实例所占用的内存空间后，还会做内存压缩。

G1垃圾回收器回收region的时候基本不会STW，而是基于 most garbage优先回收(整体来看是基于"标记-整理"算法，从局部(两个region之间)基于"复制"算法) 的策略来对region进行垃圾回收的。无论如何，G1收集器采用的算法都意味着

结果如下图：

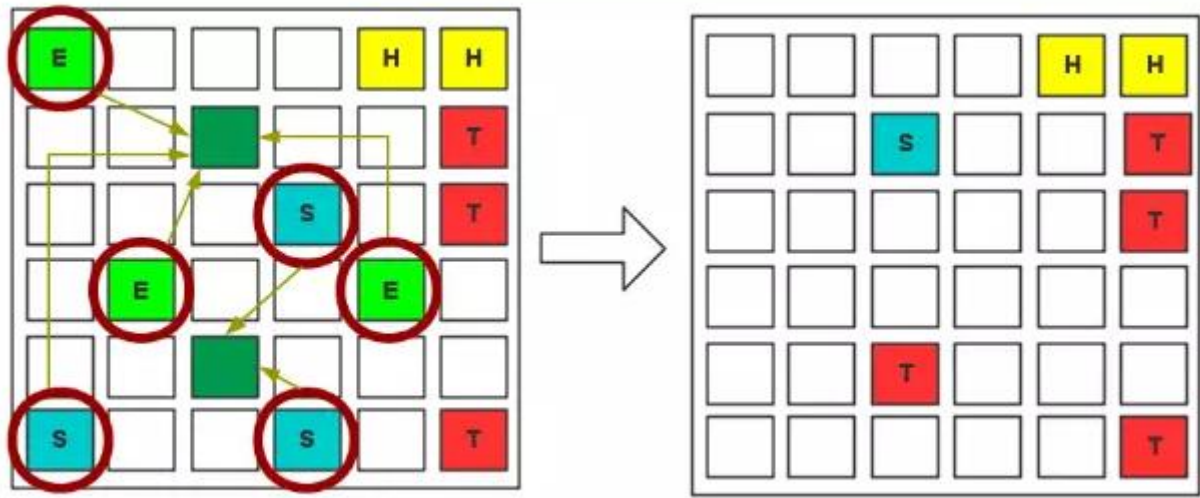


一个region有可能属于Eden，Survivor或者Tenured内存区域。图中的E表示该region属于Eden内存区域，S表示属于Survivor内存区域，T表示属于Tenured内存区域。图中空白的表示未使用的内存空间。G1垃圾收集器还增加了一种新的内存区域，叫做Humongous内存区域，如图中的H块。这种内存区域主要用于存储大对象-即大小超过一个region大小的50%的对象。

年轻代垃圾收集

在G1垃圾收集器中，年轻代的垃圾回收过程使用复制算法。把Eden区和Survivor区的对象复制到新的Survivor区域。

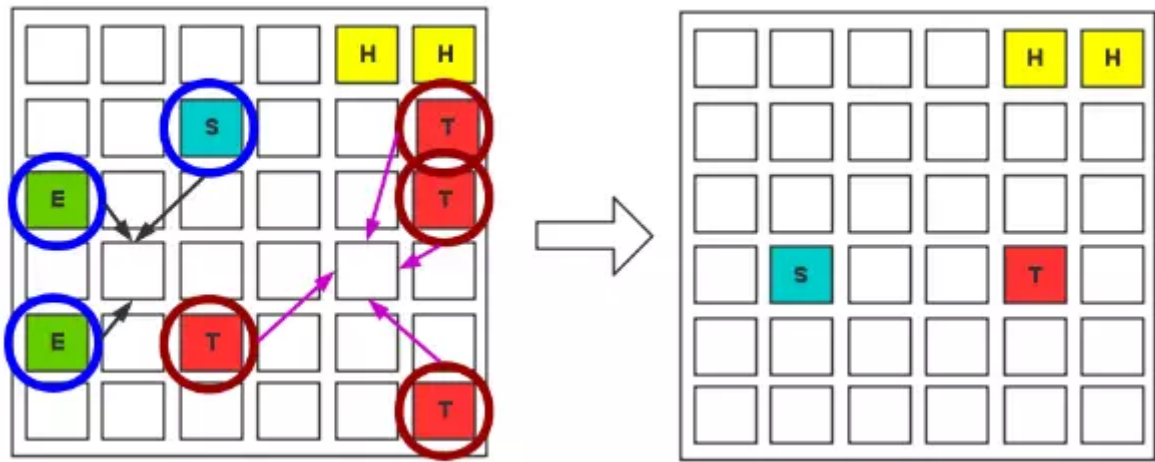
如下图：



老年代垃圾收集

对于老年代上的垃圾收集，G1垃圾收集器也分为4个阶段，基本跟CMS垃圾收集器一样，但略有不同：

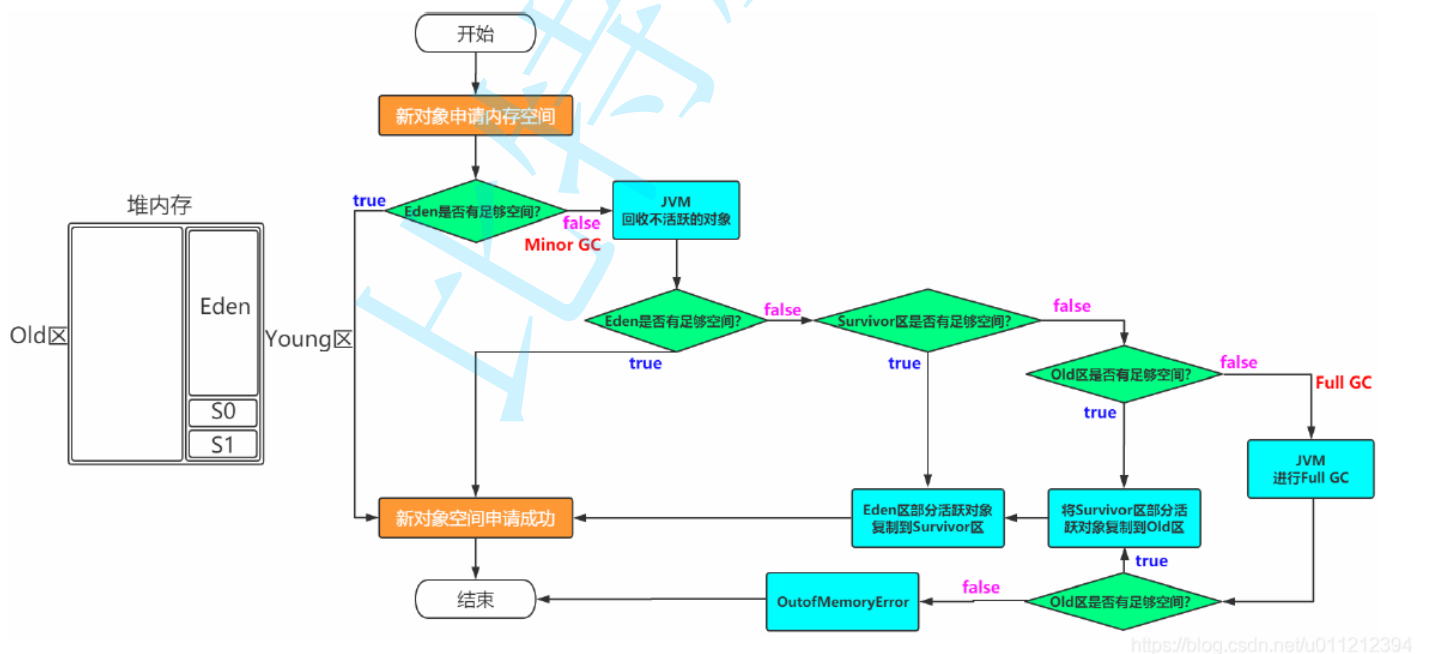
- 初始标记(Initial Mark)阶段 - 同CMS垃圾收集器的Initial Mark阶段一样，G1也需要暂停应用程序的执行，它会标记从根对象出发，在根对象的第一层孩子节点中标记所有可达的对象。但是G1的垃圾收集器的Initial Mark阶段是跟minor gc一同发生的。也就是说，在G1中，你不用像在CMS那样，单独暂停应用程序的执行来运行Initial Mark阶段，而是在G1触发minor gc的时候一并将老年代上的Initial Mark给做了。
- 并发标记(Concurrent Mark)阶段 - 在这个阶段G1做的事情跟CMS一样。但G1同时还多做了一件事情，就是如果在Concurrent Mark阶段中，发现哪些Tenured region中对象的存活率很小或者基本没有对象存活，那么G1就会在这个阶段将其回收掉，而不用等到后面的clean up阶段。这也是Garbage First名字的由来。同时,在该阶段，G1会计算每个 region的对象存活率，方便后面的clean up阶段使用。
- 最终标记(CMS中的Remark阶段) - 在这个阶段G1做的事情跟CMS一样, 但是采用的算法不同，G1采用一种叫做SATB(snapshot-at-the-beginning)的算法能够在Remark阶段更快的标记可达对象。
- 筛选回收(Clean up/Copy)阶段 - 在G1中，没有CMS中对应的Sweep阶段。相反 它有一个Clean up/Copy阶段，在这个阶段中,G1会挑选出那些对象存活率低的region进行回收，这个阶段也是和minor gc一同发生的,如下图所示：



G1 (Garbage-First) 是一款面向**服务端应用**的垃圾收集器。HotSpot开发团队赋予它的使命是未来可以替换掉JDK 1.5中发布的CMS收集器。如果你的应用追求低停顿，G1可以作为选择；如果你的应用追求吞吐量，G1并不带来特别明显的好处。

④ 总结：一个对象的一生

一个对象的一生：我是一个普通的 Java 对象，我出生在 Eden 区，在 Eden 区我还看到和我长的很像的小兄弟，我们在 Eden 区中玩了挺长时间。有一天Eden区中的人实在是太多了，我就被迫去了 Survivor 区的“From”区（S0 区），自从去了 Survivor 区，我就开始漂了，有时候在 Survivor 的“From”区，有时候在 Survivor 的“To”区（S1 区），居无定所。直到我 18 岁的时候，爸爸说我成人了，该去社会上闯闯了。于是我就去了年老代那边，年老代里，人很多，并且年龄都挺大的，我在这里也认识了很多。在老年代里，我生活了很多年（每次GC加一岁）然后被回收了。



6.JMM

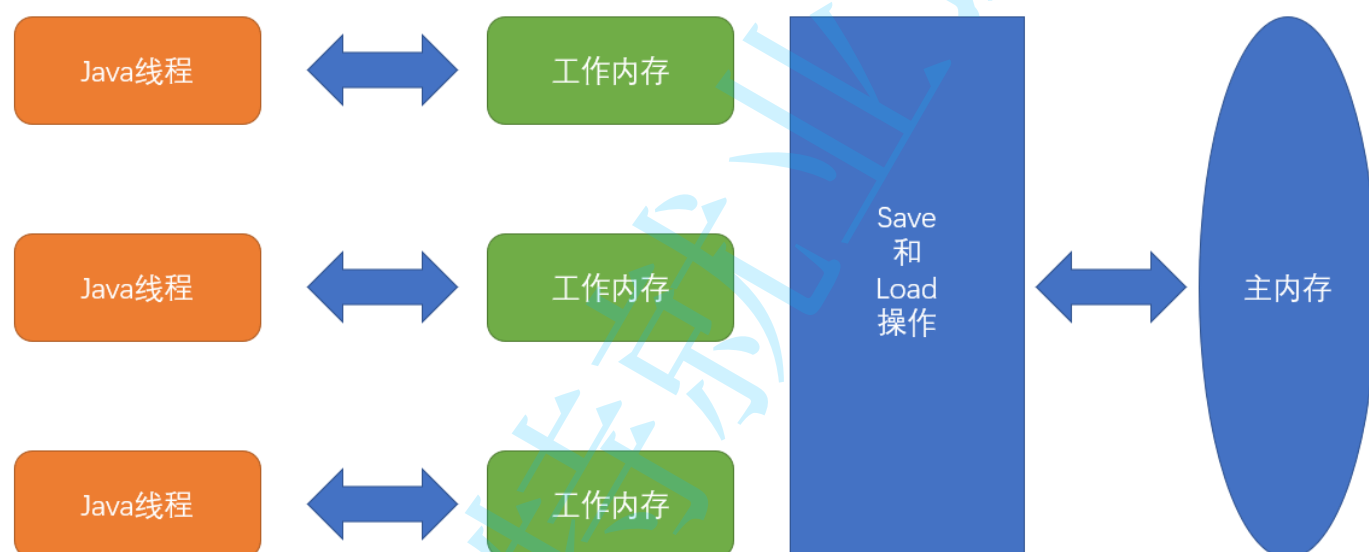
JVM定义了一种Java内存模型(Java Memory Model,JMM)来屏蔽掉各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的内存访问效果。在此之前，C/C++直接使用物理

硬件和操作系统的内存模型，因此，会由于不同平台下的内存模型的差异，有可能导致程序在一套平台上并发完全正常，而在另一套平台上并发访问经常出错。

① 主内存与工作内存

Java内存模型的主要目标是定义程序中各个变量的访问规则，即在JVM中将变量存储到内存和从内存中取出变量这样的底层细节。此处的变量包括**实例字段、静态字段和构成数组对象的元素**，但不包括局部变量和方法参数，因为后两者是线程私有的，不会被线程共享。

Java内存模型规定了所有的变量都存储在主内存中。每条线程还有自己的工作内存，线程的工作内存中保存了被该线程使用到的变量的主内存副本拷贝，线程对变量的所有操作(读取、赋值等)都必须在工作内存进行，而不能直接读写主内存中的变量。不同的线程之间也无法直接访问对方工作内存中的变量，线程间变量值的传递均需要通过主内存来完成。线程、主内存、工作内存三者的交互关系如下所示：



② 内存间交互操作

关于主内存与工作内存之间的具体交互协议，即一个变量如何从主内存中拷贝到工作内存、如何从工作内存同步回主内存之类的实现细节，Java内存模型中定义了如下8种操作来完成。JVM实现时必须保证下面提及的每一种操作的原子的、不可再分的。

- lock(锁定) : 作用于主内存的变量，它把一个变量标识为一条线程独占的状态
- unlock(解锁) : 作用于主内存的变量，它把一个处于锁定状态的变量释放出来，释放后的变量才可以被其他线程锁定。
- read(读取) : 作用于主内存的变量，它把一个变量的值从主内存传输到线程的工作内存中，以便随后的load动作使用。

- **load(载入)** : 作用于工作内存的变量，它把read操作从主内存中得到的变量值放入工作内存的变量副本中。
- **use(使用)** : 作用于工作内存的变量，它把工作内存中一个变量的值传递给执行引擎。
- **assign(赋值)** : 作用于工作内存的变量，它把一个从执行引擎接收到的值赋给工作内存的变量。
- **store(存储)** : 作用于工作内存的变量，它把工作内存中一个变量的值传送到主内存中，以便后续的write操作使用。
- **write(写入)** : 作用于主内存的变量，它把store操作从工作内存中得到的变量的值放入主内存的变量中。

Java内存模型的三大特性：

- **原子性** : 由Java内存模型来直接保证的原子性变量操作包括read、load、assign、use、store和read。大致可以认为，基本数据类型的访问读写是具备原子性的。如若需要更大范围的原子性，需要synchronized关键字约束。(即一个操作或者多个操作 要么全部执行并且执行的过程不会被任何因素打断，要么就都不执行)
- **可见性** : 可见性是指当一个线程修改了共享变量的值，其他线程能够立即得知这个修改。volatile、synchronized、final三个关键字可以实现可见性。
- **有序性** : 如果在本线程内观察，所有的操作都是有序的；如果在线程中观察另外一个线程，所有的操作都是无序的。前半句是指"线程内表现为串行"，后半句是指"指令重排序"和"工作内存与主内存同步延迟"现象。

Java内存模型具备一些先天的“有序性”，即不需要通过任何手段就能够得到保证的有序性，这个通常也称为 **happens-before** 原则。如果两个操作的执行次序无法从happens-before原则推导出来，那么它们就不能保证它们的有序性，虚拟机可以随意地对它们进行重排序。

下面就来具体介绍下happens-before原则（先行发生原则）：

- **程序次序规则**：一个线程内，按照代码顺序，书写在前面的操作先行发生于书写在后面的操作
- **锁定规则**：一个unlock操作先行发生于后面对同一个锁的lock操作
- **volatile变量规则**：对一个变量的写操作先行发生于后面对这个变量的读操作
- **传递规则**：如果操作A先行发生于操作B，而操作B又先行发生于操作C，则可以得出操作A先行发生于操作C
- **线程启动规则**：Thread对象的start()方法先行发生于此线程的每个一个动作
- **线程中断规则**：对线程interrupt()方法的调用先行发生于被中断线程的代码检测到中断事件的发生
- **线程终结规则**：线程中所有的操作都先行发生于线程的终止检测，我们可以通过Thread.join()方法结束、Thread.isAlive()的返回值手段检测到线程已经终止执行
- **对象终结规则**：一个对象的初始化完成先行发生于他的finalize()方法的开始

也就是说，要想并发程序正确地执行，必须要保证原子性、可见性以及有序性。只要有一个没有被保证，就有可能导致程序运行不正确。

③ volatile型变量的特殊规则

关键字volatile可以说是JVM提供的最轻量级的同步机制，但是它并不容易完全被正确理解和使用。JVM内存模型对volatile专门定义了一些特殊的访问规则。

当一个变量定义为volatile之后，它将具备两种特性。

第一：保证此变量对所有线程的可见性，这里的"可见性"是指：当一条线程修改了这个变量的值，新值对于其他线程来说是可以立即得知的。而普通变量做不到这一点，普通变量的值在线程间传递均需要通过主内存来完成。例如：线程A修改一个普通变量的值，然后向主内存进行回写，另外一条线程B在线程A回写完成之后再从主内存进行读取操作，新值才会对线程B可见。

关于volatile变量的可见性，经常会被开发人员误解。****volatile变量在各个线程中是一致的，但是volatile变量的运算在并发下一样是不安全的。原因在于Java里面的运算并非原子操作。**

范例：volatile变量自增操作

```
1 package com.company;
2 public class Main {
3     public static volatile int num = 0;
4     public static void increase() {
5         num++;
6     }
7     public static void main(String[] args) {
8         Thread[] threads = new Thread[10];
9         for (int i = 0; i < 10; i++) {
10             threads[i] = new Thread(new Runnable() {
11                 @Override
12                 public void run() {
13                     for (int j = 0; j < 100; j++) {
14                         increase();
15                     }
16                 }
17             });
18             threads[i].start();
19         }
20         while (Thread.activeCount() > 2) {
21             Thread.yield();
22         }
23         System.out.println(num);
24     }
25 }
```

问题就在于num++之中，实际上num++等同于num = num+1。volatile关键字保证了num的值在取值时是正确的，但是在执行num+1的时候，其他线程可能已经把num值增大了，这样在+1后会把较小的数值同步回主内存之中。

由于volatile关键字只保证可见性，在不符合以下两条规则的运算场景中，我们仍然需要通过加锁(synchronized或者lock)来保证原子性。

1. 运算结果并不依赖变量的当前值，或者能够确保只有单一的线程修改变量的值
2. 变量不需要与其他的状态变量共同参与不变约束

如下代码这类场景就特别适合使用volatile来控制并发，当shutdown()方法被调用时，能保证所有线程中执行的doWork()方法都立即停下来。

```
1 volatile boolean shutdownRequested;
2
3 public void shutdown() {
4     shutdownRequested = true;
5 }
6
7 public void work() {
8     while(!shutdownRequested) {
9         //do stuff
10    }
11 }
```

第二：使用volatile变量的语义是禁止指令重排序。普通的变量仅仅会保证在该方法的执行过程中所有依赖赋值结果的地方都能获取到正确的结果，而不能保证变量赋值操作的顺序和程序代码中执行的顺序一致。

volatile关键字禁止指令重排序有两层意思：

- 1) 当程序执行到volatile变量的读操作或者写操作时，在其前面的操作的更改肯定全部已经进行，且结果已经对后面的操作可见；在其后面的操作肯定还没有进行；
- 2) 在进行指令优化时，不能将在对volatile变量访问的语句放在其后面执行，也不能把volatile变量后面的语句放到其前面执行。

举个简单的例子：

```
1 //x、y为非volatile变量
2 //flag为volatile变量
3 x = 2;           //语句1
4 y = 0;           //语句2
5 flag = true;     //语句3
6 x = 4;           //语句4
7 y = -1;          //语句5
```


由于flag变量为volatile变量，那么在进行指令重排序的过程的时候，不会将语句3放到语句1、语句2前面，也不会将语句3放到语句4、语句5后面。但是要注意语句1和语句2的顺序、语句4和语句5的顺序是不作任何保证的。

并且volatile关键字能保证，执行到语句3时，语句1和语句2必定是执行完毕了的，且语句1和语句2的执行结果对语句3、语句4、语句5是可见的。

范例：指令重排序

```
1 Map configOptions;
2 char[] configText;
3 volatile boolean initialized = false;
4
5 //假设以下代码在线程A执行
6 //模拟读取配置文件信息，当读取完成后将initialized设置为true以通知其他线程配置可用
7 configOptions = new HashMap();
8 configText = readConfigFile(fileName);
9 processConfigOptions(configText,configOptions);
10 initialized = true;
11
12 //假设以下代码在线程B执行
13 //等待initialized为true，代表线程A已经把配置信息初始化完成
14 while(!initialized) {
15     sleep();
16 }
17 //使用线程A初始化好的配置信息
18 doSomethingWithConfig();
```

单例模式中的Double Check:

双重检验锁模式（double checked locking pattern），是一种使用同步块加锁的方法。程序员称其为双重检查锁，因为会有两次检查 `instance == null`，一次是在同步块外，一次是在同步块内。为什么在同步块内还要再检验一次？因为可能会有多个线程一起进入同步块外的 if，如果在同步块内不进行二次检验的话就会生成多个实例了。

```
1 public static Singleton getSingleton(){
2     if(instance==null){ //Single Checked
3         synchronized (Singleton.class){
4             if(instance==null){ //Double Checked
5                 instance=new Singleton();
6             }
7         }
8     }
9     return instance;
```

这段代码看起来很完美，很可惜，它是有问题。主要在于 `instance = new Singleton()` 这句，这并非是一个原子操作，事实上在 JVM 中这句话大概做了下面 3 件事情。给 `instance` 分配内存 调用 `Singleton` 的构造函数来初始化成员变量 将 `instance` 对象指向分配的内存空间（执行完这步 `instance` 就为非 `null` 了）但是在 JVM 的即时编译器中存在指令重排序的优化。也就是说上面的第二步和第三步的顺序是不能保证的，最终的执行顺序可能是 1-2-3 也可能是 1-3-2。如果是后者，则在 3 执行完毕、2 未执行之前，被线程二抢占了，这时 `instance` 已经是非 `null` 了（但却没有初始化），所以线程二会直接返回 `instance`，然后使用，然后顺理成章地报错。我们只需要将 `instance` 变量声明成 `volatile` 就可以了。

```
1 class Singleton{
2     // 确保产生的对象完整性
3     private volatile static Singleton instance = null;
4     private Singleton() {}
5     public static Singleton getInstance() {
6         if(instance==null) { // 检查对象是否初始化
7             synchronized (Singleton.class) {
8                 if(instance==null) // 确保多线程情况下对象只有一个
9                     instance = new Singleton();
10            }
11        }
12        return instance;
13    }
14 }
```